分类号_____

U D C _____

昆明理工大学

硕士学位论文

中国象棋博弈系统的研究与实现

研 究 生 姓 名 危春波

指导教师姓名、职称 王海瑞（副教授）

学 科 专 业 计算机软件与理论

研 究 方 向 人工智能

论 文 工 作

起 止 日 期 2006年10月—2008年2月

论文提交日期 2008年3月5日

中国知网 https://www.cnki.net

昆明理工大学学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下（或我个人……）进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在论文中作了明确的说明并表示了谢意。本声明的法律结果由本人承担。

学位论文作者签名：危春波

日　　期：2008年 3月 12日

关于论文使用授权的说明

本人完全了解昆明理工大学有关保留、使用学位论文的规定，即：学校有权保留、送交论文的复印件，允许论文被查阅，学校可以公布论文的全部或部分内容，可以采用影印或其他复制手段保存论文。

（保密论文在解密后应遵守）

导 师 签 名：[illegible]　论文作者签名：危春波

日　期：2008年 3月 12日　　日　期：2008年 3月 12日

中国知网 https://www.cnki.net

摘要

计算机博弈是人工智能的一个重要研究领域，对它的研究方兴未艾。本文对中国象棋计算机博弈系统中的关键技术作了研究，在此基础之上开发了一个完整的中国象棋博弈程序。论文的工作主要集中在以下几个方面：

1.文章首先研究了中国象棋在计算机中的棋盘表示问题，并针对带位行位列信息的扩展棋盘-棋子联系数组棋盘实现所有棋子的走法预生成数组，以提高搜索过程中走法产生的效率。

2.根据中国象棋的具体棋类知识讨论了如何定义一个针对局面优劣的评估机制，即估值函数；同时结合走法预生成数组提高估值的速度。

3.重点研究了$\alpha-\beta$剪枝算法及其改进策略，并针对基于迭代加深、置换表、历史启发的NegaScout综合算法进行改进；同时实现B*算法。

4.使用面向对象方法对系统进行分析与设计，并用UML语言对系统模型进行描述，最终实现系统。

该系统使用先进的棋盘表示和改进的搜索算法，在实验中证明是有效的。同时系统的整体设计与开发遵循面向对象的方法，使得系统具有良好的复用性和扩展性。

关键词：计算机博弈;棋盘表示;走法预生成;估值函数;博弈树搜索算法.

中国知网 https://www.cnki.net

ABSTRACT

Game with computer is a very important domain in AI, many production gains from it. In this paper, we make basis study on some key technique about computer Game-playing. Based on that, we design and develop an integrated Computer Came-playing system of Chinese chess. The whole work mainly focuses on the following four aspects:

1. This paper first studies how to represent Chinese chess board in computer, and discusses the advanced extend Board-Piece relative array with BitFile, BitRank information. Based on this, we generate prepared moves array of all the pieces which improved the move generation.

2. According to the character of Chinese chess, this paper discusses how to define an evaluation function which is used to compare different positions.

3. Researching on Game Tree Searching algorithm, especially on $\alpha - \beta$ pruning algorithm , and it's improved methods to master the thoughts of them. With its help a better algorithm has been proposed which base on NegaScout algorithm with transposition table, history heuristic, iterative deepening.I also inplement B* algorithm.

4. This system uses Object-Oriented method to analysis and design. Meanwhile this system uses UML to describe system model. At last, I implemented the system.

The implementation of this system adopts improved board representation and search algorithm, which is proved effective in experiment. The system's analysis, design and development are based on the Object-Oriented method, and the system is good at reusability and expansibility.

Key words: computer game-playing; board representations; moves prepare generation; evaluate function; game tree search algorithm

中国知网 https://www.cnki.net

目录

摘要 I

ABSTRACT II

第一章 绪论 1

1.1 选题背景 1

1.2 国内外研究现状 2

1.3 课题的主要工作和章节安排 4

1.3.1 课题的主要工作 4

1.3.2 章节安排 5

第二章 棋盘表示和走法生成 6

2.1 棋盘表示（Board Representations） 6

2.1.1 基本表示法 6

2.1.2 改进型棋盘 7

2.2 走法产生(Move Generation) 10

2.2.1 走法产生伪代码及效率分析 10

2.2.2 着法预生成 11

2.2.3 合理走法的产生 15

2.2.4 人类走法合理性判断 15

2.3 小结 16

第三章 局面估值 17

3.1 估值函数(Evaluation Function) 17

3.1.1 棋子固定价值 17

3.1.2 棋子位置附加值 18

3.1.3 棋子灵活性 18

3.1.4 棋子对棋盘的控制 18

3.1.5 棋子间的关系 19

3.2 估值的速度与博弈性能 19

3.3 估值与走法预生成数组相结合 20

中国知网 https://www.cnki.net

第四章 博弈树搜索技术 21

4.1 博弈树（Game Tree） 21

4.2 极小极大值算法（Minimax Algorithm） 22

4.3 $\alpha-\beta$ 剪枝算法及其改进策略 25

4.3.1 $\alpha-\beta$ 剪枝算法(Alpha-Beta Pruning Algorithm) 25

4.3.2 窗口原则(Window principle) 28

4.3.3 置换表(Transposition Table)与哈希表（Hash Table） 30

4.3.4 历史启发（History Heuristic） 33

4.3.5 迭代深化（Iterative Deepening） 34

4.3.6 其他问题 35

4.3.7 各种改进策略作用于 $\alpha-\beta$ 剪枝算法的实验数据 36

4.4 综合搜索算法及其改进 37

4.5 B*算法 43

第五章 系统设计 46

5.1 面向对象的分析与设计简介 46

5.2 UML 简介 47

5.3 类图 49

5.4 系统的实现及用户界面操作 49

第六章 实验与结果 50

6.1 程序说明 50

6.2 实验结果及分析 50

第七章 总结与展望 59

7.1 工作总结 59

7.2 今后的工作 59

致谢 61

参考文献 62

攻读硕士学位期间发表的文章 65

中国知网 https://www.cnki.net

第一章 绪论

1.1 选题背景

在人工智能(AI，Artificial Intelligence)领域内，博弈(GP,Game Playing)被认为是最具挑战性的研究方向之一[1]。到目前为止，由博弈衍生而来的技术广泛应用于政治、经济、军事和生物竞争等领域中，并取得大量引人瞩目的成果。因此博弈在各种文献中被誉为人工智能的果蝇。

博弈问题无所不在，小到孩童的游戏与争论、各种场合下的讨价还价，大到商家的竞争、各种突发事件(恐怖、灾害)的应急处理、国家的外交、流血的和不流血的战争，只要局中的双方主体存在某种利益冲突，博弈便成为矛盾表现和求解的一种方式[1]。

人工智能中对博弈规律的研究主要集中在棋类（如国际象棋、中国象棋、围棋、五子棋、西洋跳棋等）。早在上世纪五十年代，就有人设想利用机器智能来实现机器与人的对弈。国内外许多知名学者和知名科研机构都曾经涉足这方面的研究，历经半个多世纪，到目前为止已经取得了许多惊人的成就。其中，国际象棋的计算机博弈已经有了很长的历史，并且经历了一场波澜壮阔的“搏杀”，据不完全统计，先后参与国际象棋计算机博弈的计算机公司多达数百家，而代表队则数以千计[2]。1997年IBM的“深蓝”战胜了国际象棋世界冠军卡斯帕罗夫，更是惊动了世界[3]。然而，面对历史悠久的中国象棋的计算机博弈，不仅显得有些稚嫩，而且好像在学术界少有人问津。计算机与人工智能学科最具挑战性的课题之一成为“被爱情遗忘的角落”[1]。

在计算机日益普及和大众化的现代社会，高水平的博弈系统很容易获得可观的商业价值。目前，世界领先的计算机国际象棋程序和中国象棋程序基本上都是商业产品。事实上，个人计算机软件市场的大约80%销售份额是来自游戏软件[3]，其中有传统的博弈游戏，而非博弈游戏中也不可缺少人工智能与博弈的成分。因此，对游戏开发过程中的人工智能技术的研究自然也就成了业界的一个热门研究方向。

针对上面各种情况，且本人对人工智能，游戏开发比较感兴趣。希望能在国际象棋成熟技术的基础上，结合在中国象棋机器博弈方面的多年实践，实现一个具有一定智能和棋力的中国象棋计算机博弈系统。或可给研究人工智能机器博弈的同仁提供一点的参考资料和意见；也为我今后从事游戏软件开发，特别是与博

中国知网 https://www.cnki.net

弈系统相关的系统的开发打下一定的基础。

1.2 国内外研究现状

和国际象棋博弈系统相比，中国象棋博弈系统的研究起步较晚，是八十年代开始的。在这个时候计算机国际象棋取得重大突破， 1983年美国贝尔公司的电脑参加美国人类比赛，成绩优异，等级分达2300分，被授予大师称号，于是有人想到如将国际象棋电脑技术移植到中国象棋上来，以带动中国象棋电脑较快的发展；同时中国象棋从技术研究进入理论研究，有关开、中局、残局理论以及象棋对策相继建立起来，为中国象棋计算机博弈提供了理论基础。

1981年张耀腾发表的《人造智慧在电脑象棋上的应用》，他提出审局函数为静态子力值，棋子位置值，棋子灵活度值，威胁与保护等四项之和。但该文主要以残局做实验，缺乏完整对局的考虑。1982年廖嘉成发表的《利用计算机象棋的实验》就进了一步，包括开局、中局、残局。1983年黄少龙、周玉龙合作制成《象棋排局系列软盘》专家系统与人对弈。

近二十年来，中国象棋博弈系统的研究常常通过比赛对弈的方式来验证各自研究的成果，当前的主要比赛包括:1989年起由ICGA (ICGA,International Computer Games Association)每年举办的Computer Olympiad（奥林匹克计算机游戏程序竞赛）；2004年起在台湾成功大学举办的世界电脑象棋争霸赛；2006年8月起全国计算机博弈锦标赛。在这些赛事上，来自海内外的专家学者及18支代表中国象棋计算机博弈最高水平的参赛队伍会聚一堂，进行比赛和学术交流。这些比赛极大的推动了中国计算机博弈的普及和发展。

目前知名的象棋软件有："棋天大圣"，"象棋奇兵"，"象眼（ElephantEye）"，"纵马奔流"，"谢谢大师"等十余种，其中一些已经接近或达到大师级水平。其中"棋天大圣"是东北大学人工智能与机器人研究所从2003年开始研制的象棋软件，它聘请"深蓝之父"— 徐峰雄博士为其顾问，获得第十一、十二届电脑奥赛金牌和首届全国计算机博弈锦标赛冠军。"象眼（ElephantEye）"是一款优秀的电脑象棋引擎，是黄晨个人作品，是公开源代码的（基于GUN公司的GPL授权），可用于学习与交流。本系统的实现在棋盘表示上参考"象眼"这款优秀的象棋引擎。

目前中国象棋博弈系统的研究热点与难点集中在如下几个方面[1,5]：

1.棋盘表示（Board Representations）

棋盘表示就是使用一种数据结构来描述棋盘及棋盘上的棋子，以方便计算机

中国知网 https://www.cnki.net

处理。中国象棋的棋盘表示还没有形成统一的或者公认哪种为最佳的数据格式。最直观也是最典型的方式是使用10×9的二维棋盘数组表示，也有使用棋子数组，扩展棋盘—棋子数组等，棋盘的数据表示直接影响到程序的时间及空间复杂度。

2.着法生成（Move Generation）

着法生成就是找到某个局面所有合法的走法。它与棋盘表示的数据结构密切相关，一般需要大量繁琐的判断，伴随着搜索进行，并且调用频繁，是相当复杂而且耗费运算时间。在一定程度上形成了程序的性能瓶颈。当前为了提高着法生成的效率，通常采用以空间换时间的方法：与先求出棋子的在某位置的所有走法。

3.评估函数（Evaluation Function）

评估函数就是对博弈过程中实际局面的好坏做出判断或打分，它影响着博弈局势发展的趋势。目前象棋程序的静态评价函数主要有子力价值，子力灵活性，子力对棋盘的控制，和一些对棋局影响比较大的重要特征计算几部分组成。它与着法生成一样十分耗费运算时间，如何加速评估函数的速度十分关键。

4.搜索技术（Search Techniques）

搜索技术与算法是象棋博弈系统程序的核心，是决定搜索效率的关键所在。再好的硬件也无法实现“象棋不败算法”。如何在成指数递增的状态空间中寻求最优解，是象棋博弈系统的一个重要的方向。“蛮力搜索”肯定是不可取的。如何有选择地搜索，既瞄准最有希望的方向局部加深，又不遗漏其它存在最优解的可能。目前主要是使用$\alpha-\beta$剪枝算法加上迭代深化、置换表、历史启发等策略的综合应用。

5.开局库（Opening Book）

把开局几步的走法建成数据库供程序直接取用。

实践表明无论搜索引擎如何出色，在开局搜索过若干层后，得到的可能只是一些贻笑大方的开局。直接从开局库中取就可以大大加快开局时的运行速度和提高开局的质量。开局库一般是采用zobrist哈希技术加以实现。

6.残局库（Endgame Database）

残局是指由少数残余子力所构成并进入决定胜负阶段的对峙局面。残局库就是把此时局面的评估值保留以备查找，如果将象棋残局库中的局面所包含的棋子增加到32子，那么象棋将被彻底解决（6子残局库需要的硬盘空间大约是20G）。现在的残局库多是用回溯分析的方法生成的巨大数据库。由于残局库信息量庞大，对残局数据库的索引方案和存储压缩，都是不容忽视的任务。

中国知网 https://www.cnki.net

7.机器自学习（Machine Learning）

机器自学习之一是针对评估函数。对弈过程机器自动调整评估函数参数的权值，进行优化，发现一些棋子之间潜在的联系；之二是采用模式识别，学习的过程是通过对博弈过程的识别统计，自行丰富模式库中的内容，以提高程序的博弈性能。

中国象棋计算机博弈的研究者在国际上所达到的水平还不如国际象棋和西洋跳棋等棋类那样成功。为此中国人工智能学会郑重发出号召，积极参加计算机博弈研究，让中国象棋的计算机博弈水平迅速提高，逐步接近大师和特级大师级的水平，有朝一日也能像“深蓝”计算机一样，最终战胜中国象棋的全国冠军和世界冠军[2]。

1.3 课题的主要工作和章节安排

1.3.1 课题的主要工作

本课题的主要工作是将博弈策略应用到中国象棋程序的设计与实现上，建立一个完整的中国象棋计算机博弈系统，研究工作主要集中在以下几个方面：

(1) 棋盘表示与走法生成

从操作速度（性能）以及使用方便与否考虑，本象棋博弈系统从带位行位列信息的扩展棋盘-棋子联系数组着手进行研究。然后根据棋盘表示获得所有棋子的走法预生成数组。在产生走法时直接从中取出数据，进行少量判断以得到该棋子的合法走步。

(2) 估值函数

如何快速有效的对一个局面进行评估需要从速度和知识的两个角度进行考虑：一般知识越多估值越准确，但速度也慢下来。本系统采用通用的静态估值方式，同时在估值时结合走法预生成数组，使估值的速度有很大提升。

(3) 搜索技术

博弈树搜索技术它很大程度上独立于具体的计算机博弈程序。本文讨论了$\alpha-\beta$剪枝搜索算法及其各种增强手段：包括窗口原则(PVS，渴望搜索，MTD(f))；置换表（内存增强）；历史启发表（节点顺序调整）；迭代深化（时间控制）等。如何把各种增强手段有机组合起来达到最优，文章对基于迭代加深、置换表、历史启发的NegaScout算法进行改进。同时实现B*算法[4]。

(4) 系统设计

中国知网 https://www.cnki.net

本系统采用面向对象方法对系统进行分析与设计。介绍了面向对象的分析与设计以及UML建模工具,并给出系统的类图。实现界面基本功能：先后手， 悔棋，换色， 机器之间的对弈等。

1.3.2 章节安排

第一章阐述了选题背景，课题的国内外研究现状及课题的主要工作和文章的章节安排。

第二章首先介绍了中国象棋的棋盘表示，着重研究了带位行位列信息的扩展棋盘-棋子联系数组棋盘；在此基础上实现所有棋子的走法预生成数组，以提高搜索过程中走法产生的效率。

第三章描述本系统的评价函数架构，包括子力分数、子力灵活度评价、棋盘控制等，并与走法预生成数组结合以提高估值速度。

第四章研究了博弈树的$\alpha-\beta$剪枝算法及其各种增强策略，并对迭代加深、置换表、历史启发的NegaScout综合算法进行改进；同时实现B*算法。

第五章讨论了系统的设计，给出系统的类图。

第六章给出实验环境和实验结果。

第七章是全文的总结及展望。

中国知网 https://www.cnki.net

第二章 棋盘表示和走法生成

2.1 棋盘表示（Board Representations）

棋盘表示就是使用一种数据结构来描述棋盘上的信息，以便程序知道博弈的状态。棋盘的数据表示直接影响到程序的时间及空间复杂度。为了追求更高效率，研究人员针对中国象棋提出了多种不同的表示方法。

2.1.1 基本表示法

中国象棋的棋盘表示中最典型的是用一个10×9的二维字节（byte）数组，数组中每个元素代表棋盘上的一个交点，其值表明这个交点上放置的是一个什么棋子或是没有棋子[5]（如图 2.1）。

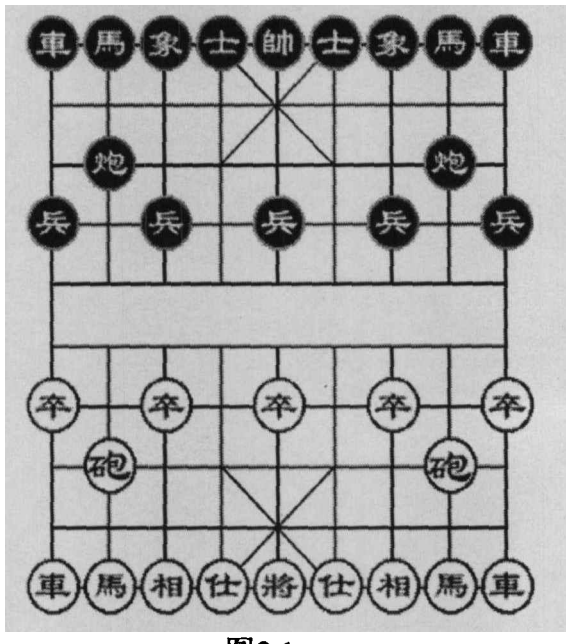


图2.1

```
const byte Position[10][9] =
{ {2,  3,  6,  5,  1,  5,  6,  3,  2},
  {0,  0,  0,  0,  0,  0,  0,  0,  0},
  {0,  4,  0,  0,  0,  0,  0,  4,  0},
  {7,  0,  7,  0,  7,  0,  7,  0,  7},
  {0,  0,  0,  0,  0,  0,  0,  0,  0},

  {0,  0,  0,  0,  0,  0,  0,  0,  0},
  {14,  0, 14,  0,  14, 0,  14, 0,  14},
  {0,  11,  0,  0,  0,  0,  0, 11,  0},
  {0,  0,  0,  0,  0,  0,  0,  0,  0},
  {9, 10,  13, 12,  8, 12,  13,  10, 9},
};//1~7分别表示帅，車，馬，炮，士，象，兵。
//8~14分别表示将，車，馬，跑，仕，相，卒。
//0表示没有棋子
```

从棋子的角度考虑，如果把棋盘看成是一个平面坐标系，可以知道每一个棋子的位置信息：小于 10 的横坐标和小于 11 的纵坐标；又在棋盘上最多 32 个棋子，故可以用一个 32 个字节的一维数组表示所有 32 个棋子的位置，其中每个字节的高 4 位表示该棋子的横坐标，低 4 位表示棋子的纵坐标。而已被吃掉的棋子用坐标范围以外的数表示。这样棋盘信息就被装入这 32 个字节中。当然也可以把棋盘看作一维的，每个元素保存直接的位置信息。

用 32 个字节表示棋盘，数据结构极为紧凑，适用于掌上电脑和手机上编程，可以赢得空间上的优势。但在随着计算机存储能力大幅度提高，空间需求已成为

中国知网 https://www.cnki.net

次要问题，而在程序进行计算操作的时候大部分时间需要先移位取出棋子的坐标信息，使用完又必须先移位保存到字节中去；把棋盘看成一维数组时求坐标时也需要取整（获得纵坐标）取余（获得横坐标），这极大的影响了效率，同时不容易理解和使用，故在PC机上很少使用了。

上面介绍的两种棋盘表示方法：一是做一个棋盘数组；二是做一个棋子数组。棋盘数组中由棋子的位置获得棋子的类型可以在常数时间内完成，但由棋子的类型获得棋子的位置需要遍历；在棋子数组中由棋子的类型获得棋子的位置可以在常数时间内完成，但由棋子的位置获得棋子的类型操作繁琐。如果一个程序同时使用这两个数组，那么获得棋子的类型和棋子的位置都可以在常数时间内完成。这就是“棋盘-棋子联系数组”，它的技术要点是：

(1) 同时用棋盘数组和棋子数组表示一个局面，棋盘数组和棋子数组之间可以互相转换。

(2) 随时保持这两个数组之间的联系，棋子移动时必须同时更新这两个数组。

根据这两个原则，棋盘-棋子联系数组可以定义为：

```
struct PositionStruct {
  int Squares[90];  //数组的下标表示棋盘的位置，数组元素的值记录该位置棋子的类型(包括空格);
  int Pieces[32];   //数组的下标记录棋子的类型，数组元素的值记录该棋子的位置(包括被吃的状态);
};//之所以使用长为90的一维数组而不使用10×9的二维数组是因为：
  //1.代码更直观。2.避免在棋子数组的定位时除以10和对9取余操作。
```

在棋盘上删除一个棋子，需要做两个操作(分别修改棋盘数组和棋子数组)。同样，添加一个棋子时也需要两个操作。

“棋盘-棋子联系数组”最大的优势是：对于着法产生过程，可以逐一查找棋子数组，如果该子没有被吃掉，就产生该子的所有合理着法，由于需要查找的棋子数组的数量(每方只有16个棋子能走)比棋盘格子的数量(90个格子)少得多，因此联系数组的速度要比单纯的棋盘数组快很多。同时“棋盘-棋子联系数组”是很多改进的棋盘的基础。

2.1.2 改进型棋盘

在计算机上面，位运算比一般的加减乘除及取余运算快很多。如果能在寻找棋子和定位棋子上使用位运算代替加减乘除和取余，这将很大程度上提高运算速

中国知网 https://www.cnki.net

度。基于这种想法，人们把“棋盘-棋子联系数组”进行扩展（使用数组ucpcSquares[256]和 ucsqPieces[48]）：把棋盘做成 16x16 的大小，这样得到行号可以用 16 除（右移 4 位），得到列号可以对 16 取余（和 15 进行与运算），这比除以 10 和对 9 取余操作要快得多。用下面的结构表示（底色是红色的格子都被标上“出界”的标记，目标格在这些格子上就说明着法无效）：

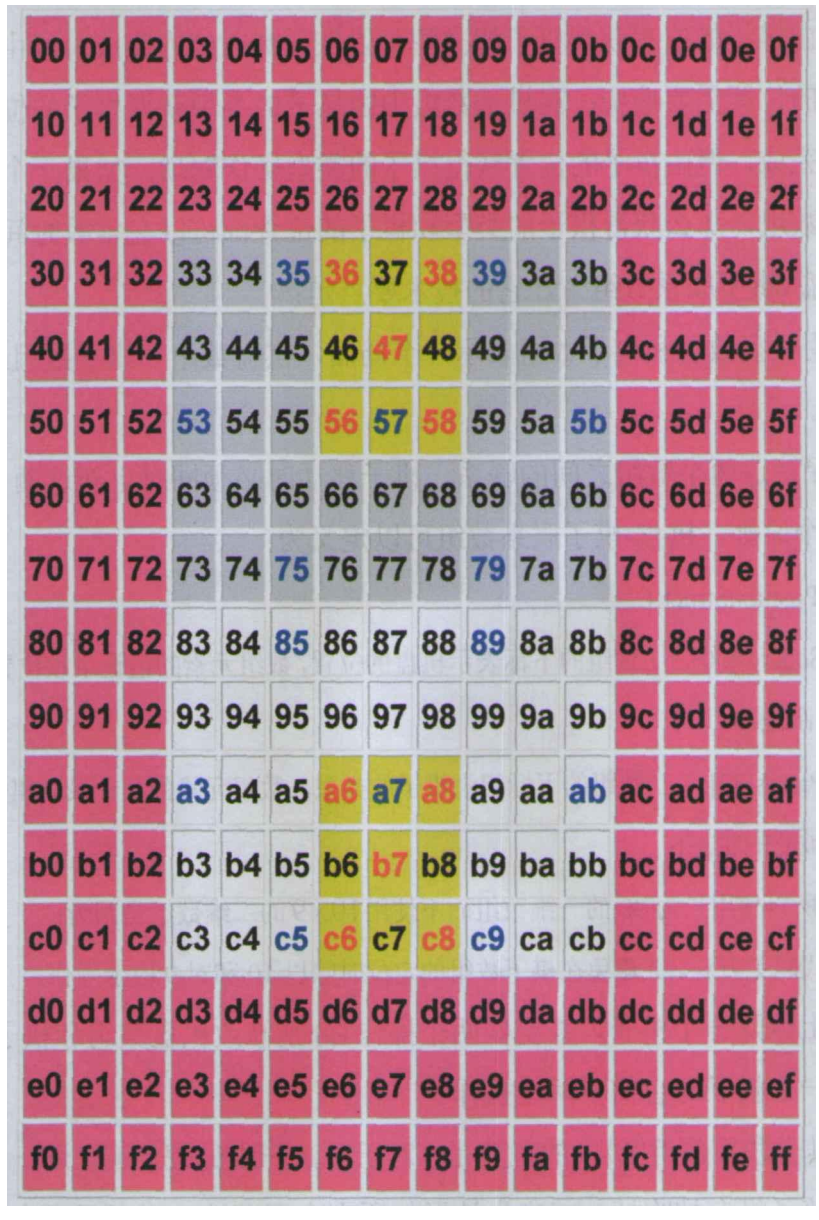


图 2.2 扩展的棋盘

16x16 的棋盘还有更大的好处：

1.它可以防止棋子走出棋盘边界。由 bool 型棋盘数组 cbcInBoard[256]（属于棋盘的位置为 true,否则为 false),判断棋子是否走出棋盘边界只要返回该数组对应的值即可，速度非常快！

2. 对于是否在城池内可以用 bool 型城池数组（城池内的位置置为 true,其他位置置为 false）判断，由于是否在城池内的判断会非常频繁故直接用该数组会很

中国知网 https://www.cnki.net

高效。

3.判断是否过河的方法非常简单： 表达式(SrcSq^DstSq & 0x80)为假表示已过河，否则没过河，其中 sqSrc 表示所在棋子的起始位置，sqDst 表示棋子的目标位置。

4.对于是否在己方可以如下判断：(sq & 0x80) == (sd << 7)，其中 sq 表示棋子所在位置，sd 表示哪一方（红方为 0，黑方为 1）（详细实现请参考：Pregen.h）。

Pieces[48]这个扩展的棋子数组实际上用了"屏蔽位"的设计，即 1 位表示红子(16)，1 位表示黑子(32)。因此 0 到 16 没有作用，16 到 31 代表红方棋子(16 代表帅，17 和 18 代表车，依此类推，直到 27 到 31 代表兵)，32 到 47 代表黑方棋子(在红方基础上加 16)。这样，棋盘数组 Squares[256]中的每个元素的意义就明确了，0 代表没有棋子，16 到 31 代表红方棋子，32 到 47 代表黑方棋子。这样表示的好处就是：它可以快速判断棋子的颜色，(Piece & 16)可以判断是否为红方棋子，(Piece & 32)可以判断是否为黑方棋子。判断两个棋子是否一伙可以用(Piece1 & 16)等于(Piece2 & 16) 。

20 世纪 60 年代末前苏联的 KAISSA 项目组提出比特棋盘。国际象棋利用此技术在 64 位主机上使运算速度取得很大的飞跃。它用 64 位数表示一个棋子的位置：每个位置用一个比特位表示，这样一个国际象棋棋盘上的全部信息可以用 12 个 64 位数（比特棋盘）表示。使用时先利用数据库技术把棋子在某一位置可以到达的位置用一个比特棋盘保存，求合法走法时只需要先根据具体的局势计算得到不可走的比特棋盘，然后与可走的比特棋盘与运算即可得到合法的走法。

然而，中国象棋棋盘不是 8×8，且大多数人使用的 PC 机是 32 位的处理器。为了借鉴比特棋盘的思想，人们在不断探索中形成了位行位列技术，但它仅限于车和炮的着法，在"象眼"和"纵马奔流"象棋软件中都用到了该技术。它以"位"的形式记录棋盘上某一行所有的格子的状态(仅仅指是否有子)，就称为"位行"(BitRank)，与之对应的是"位列"(BitFile)，棋盘结构应该包含 10 个位行和 9 个位列，即：

```
struct PositionStruct {
    ......//其他成员
    unsigned short wBitFiles[16]; //每个元素只用了前面的 10 位
    unsigned short wBitRanks[16]; //每个元素只用了前面的 9 位
};
```


中国知网 https://www.cnki.net

它的运作方式有点和“棋盘-棋子联系数组”类似：

(1) 同时用位行数组和位列数组表示棋盘上的棋子分布信息，位行数组和位列数组之间可以互相转换；

(2) 随时保持这两个数组之间的联系，棋子移动时必须同时更新这两个数组。

位行位列技术能帮助程序以查表的方式快速获得车和炮的所有走法（在走法产生中继续介绍）！

综上，本系统使用棋盘数据结构如下：

```
struct PositionStruct
{   int sdPlayer;                  // 哪一方走棋，红为 0，黑为 1
    uint8 ucpcSquares[256];   // 元素的值表示每个格子放的棋子，0 表示没有棋子
    uint8 ucsqPieces[48];     // 元素的值表示每个棋子放的位置，0 表示被吃
    uint16 wBitRanks[16];     // 位行数组，
    uint16 wBitFiles[16];     // 位列数组，
}
```

注：带位行位列信息的扩展棋盘-棋子联系数组棋盘在论文的以下部分称做带位行位列信息的扩展棋盘

2.2 走法产生(Move Generation)

走法产生包括根据博弈的规则产生所有合法走法，并判断人类对手是否按规则走棋。走法产生是博弈程序中一个相当复杂而且耗费运算时间的方面。不过，通过良好的数据结构和走法预生成，可以显著地提高生成的速度。

2.2.1 走法产生的效率分析

以兵为例求走法：

```
void Gen_PawnMove(Position p, int pos)
{ if(move forward&&InBoard&& do not touch self-piece ) //前行在棋盘上且没有本方棋子阻挡
    AddMove;                  //把该走法加入合法走步队列
  if(have crossed river)      //棋子已经过河
  {  if(move right && InBoard &&do not touch self-piece) //右行在棋盘上且没有本方棋子阻挡
        AddMove;              //把该走法加入合法走步队列
     if(move left && InBoard &&do not touch self-piece ) //左行在棋盘上且没有本方棋子阻挡
```


中国知网 https://www.cnki.net

```
            AddMove;          //把该走法加入合法走步队列
        }
    }
```

求走法时首先需要遍历棋子数组来找到相应的棋子，然后根据棋子类型来求走法；而仅仅一个兵的走法就需要做出若干判断才可以找到合法的走步。而在中国象棋中，兵的走法还是比较容易判断的。在程序执行过程中，走法产生通常伴随搜索进行，调用是非常频繁的，使用极小极大值算法在搜索四层就可达到平均将近三百万个节点，这在一定程度形成程序的性能瓶颈。

2.2.2 着法预生成

为了提高走法产生的效率，通常的做法是把每种棋子在某一位置的最大可走步建成一个数据库，在产生走法时直接取出数据，然后根据具体的棋局去除不合法的走法，这时候判断要少很多！这是典型的以空间换时间优化！

下面结合上节的局面表示来求着法预生成数组。

先求短程棋子（包括将，相，士，马，兵）的着法预生成数组。以马为例：马有 8 个着法，用数组 cnKnightMoveTab[8] = {-0x21, -0x1f, -0x12, -0x0e, +0x0e, +0x12, +0x1f, +0x21}表示着法表（增量），对应的马腿表就是 cnHorseLegTab[8] = {-0x10, -0x10, -0x01, +0x01, -0x01, +0x01, +0x10, +0x10}；为了使马在每个位置都知道它最大的可走步，定义 256×12，256×8（ucsqKnightMoves[256][12]和 ucsqHorseLegs[256][8]）两个二维数组：256 表示棋盘上的位置，12 代表在该位置可走的最大步，之所以大于 8 是因为着法生成器依次读取数组中的值，读到 0 就表示不再有着法(12 则是为了对齐地址)。下面用伪代码来描述这一过程。

```
//获得马的着法预生成数组（包括马腿数组）
void GetMoveTabOfHorse(void)
{   int sqSrc, sqDst，n;//马的起始和目标位置及可达位置个数
    for (sqSrc = 0; sqSrc < 256; sqSrc ++) //对每个位置
    {   n = 0;
        if (IN_BOARD(sqSrc)) //如果在棋盘上
        {   for (i = 0; i < 8; i ++) //对应马的 8 个着法
            {   sqDst = sqSrc + cnKnightMoveTab[i];//计算可达位置
                if (IN_BOARD(sqDst))
```


中国知网 https://www.cnki.net

```
                {   ucsqKnightMoves[sqSrc][n] = sqDst;//保存可达位置
                    //计算马腿的位置并保存
                    ucsqKnightPins[sqSrc][n] = KNIGHT_PIN(sqSrc, sqDst);
                    n ++;
                }
            }
            //把最后一个位置置为 0 以便确定循环次数(查找最大可达位置个数时)
            ucsqKnightMoves[sqSrc][n] = 0;
        }
    }
}
```

同理，帅（将），士（仕），象（相），兵（卒）可以仿照马的生成着法预生成数组如法炮制。其中帅（将）的着法表为 cnKingMoveTab[4] = {-0x10, -0x01, +0x01, +0x10};士(仕)的着法表为 cnAdvisorMoveTab[4] = {-0x11, -0x0f, +0x0f, +0x11};象（相）的着法表为 cnBishopMoveTab[4] = {-0x22, -0x1e, +0x1e, +0x22}，象眼的计算可以用如下表达式：(sqSrc + sqDst) >> 1；红兵的着法表为:过河前{-0x10}，过河后{-0x10,-0x01,+0x01}，黑卒反过来。但帅（将）和士（仕）要考虑在没在城池内；象（相）要考虑走步是否过河；兵（卒）要考虑棋子是否在己方，且红黑两方着法预生成数组不同。

对于远程棋子（包括车和炮）的着法预生成数组不能按上面的方法生成，因为它们没有固定的着法表，只能根据它们所在行与列的实际情况来确定。所以引入位行和位列来表示棋盘上棋子分布情况，然后根据棋子在某一行和某一列的棋子分布确定它们的走法。这样对于某一特定的位行或位列（一行或一列的棋子分布），需要知道车和炮在不同位置可以到达的位置以及吃子可达的位置。对吃子可达的位置，先假定先不管吃到的是己方棋子还是敌方棋子，这样就只取决于它所在的行和列上的棋子分布，而跟棋子的颜色和兵种无关，因此这些信息完全反映在位行和位列中。可以用下面的结构来描述车和炮的位行或位列着法的预置结构：

```
struct SlideMoveStruct {
  uint8 ucNonCap[2]; //不吃子能走到的最大一格/最小一格,车和炮是一样的
  uint8 ucRookCap[2];    // 车吃子能走到的最大一格/最小一格
```


中国知网 https://www.cnki.net

```
uint8 ucCannonCap[2]; // 炮吃子能走到的最大一格/最小一格
```

};然后预置两个数组，以位行或位列为指标查找数组，就可以立即知道车或炮能到那个位置。如下：

```
SlideMoveStruct smvRankMoveTab[9][512];   //车炮横向上的着法表
SlideMoveStruct smvFileMoveTab[10][1024]; //车炮纵向上的着法表
```

说明：以 smvRankMoveTab[9][512]来解释，9 表示车或炮在某一行的位置数， $512=2^9$ 表示在某一行棋子有无的排列数。smvFileMoveTab[10][1024]的解释与此类似。当然里面有一些组合是没有意义的，因为不可能出现,故存在内存浪费。

下面用伪代码描述如何产生车炮的着法预生成数组：

```
void GetMoveTabOfRemoate（void）//获得车炮的着法预生成数组
{   int i, j, k;
    SlideMoveStruct smv; //着法预生成结构临时变量
    //先求车炮横向着法预生成数组: smvRankMoveTab[9][512]
    for (i = 0; i < 9; i ++) //某行的九个位置
    {   for (j = 0; j < 512; j ++) //某一行棋子有无的排列数
        {   //初始化临时变量 smv，假设没有着法，用起始格填充
            InitSMV(smv, i);
            //考虑向右移动的目标格，填充...[0],
            for (k = i + 1; k <= 8; k ++)
            {   if ((j & (1 << k)) != 0) //此时碰到棋子：车吃子
                {   smv.ucRookCap[0] = FILE_DISP(k + FILE_LEFT);//车所能吃到的
                    最右边棋子（格）
                    break;
                }
                //不吃子时，车与炮能到达的棋格一样
                smv.ucNonCap[0] = FILE_DISP(k + FILE_LEFT);
            }
            //考虑炮吃子
            for (k ++; k <= 8; k ++)
            {   if ((j & (1 << k)) != 0) //此时碰到棋子：为炮吃子
                {smv.ucCannonCap[0] = FILE_DISP(k + FILE_LEFT);
```


中国知网 https://www.cnki.net

```
                    break;
                }
            }
            //考虑向左移动的目标格，填充...[1]
            ……//向左移动目标格与向右移动目标格思路是一样的
            //将临时变量 smv 拷贝到着法预生成数组中
            smvRankMoveTab[i][j] = smv;
        }
    }
    // 然后生成车炮纵向的预置数组: smvFileMoveTab[10][1024]
    for (i = 0; i < 10; i ++) //某列的 10 个位置
    {   for (j = 0; j < 1024; j ++) //某列棋子有无的排列数
        {
            InitSMV(smv, (i + 3) * 16);
            ……//生成车炮纵向的预置数组基本思路与横向是一致的
            smvFileMoveTab[i][j] = smv;
        }
    }
}
```

综上，总的着法预生成数组对应的数据结构如下：

```
struct PreGenStruct {
  uint16 wBitRankMask[256]; // 每个格子位行的屏蔽位,用于位行数组的快速修改操作
  uint16 wBitFileMask[256]; // 每个格子位列的屏蔽位，用于位列数组的快速修改操作
  SlideMoveStruct smvRankMoveTab[9][512]; //车炮横向的着法预生成数组
  SlideMoveStruct smvFileMoveTab[10][1024]; //车炮纵向的着法预生成数组
  uint8 ucsqKingMoves[256][8]; //将（帅）的着法预生成数组
  uint8 ucsqAdvisorMoves[256][8]; //士（仕）的着法预生成数组
  uint8 ucsqBishopMoves[256][8]; //象（相）的着法预生成数组
  uint8 ucsqBishopPins[256][4]; //对应的象（相）眼的预生成数组
  uint8 ucsqKnightMoves[256][12]; //马（馬）的着法预生成数组
  uint8 ucsqKnightPins[256][8]; //对应的马（馬）腿的预生成数组
```


中国知网 https://www.cnki.net

```
    uint8 ucsqPawnMoves[2][256][4]; //对应的兵（卒）的着法预生成数组
};
```

上面探讨了如何获得中国象棋的所有（七种）棋子的走法预生成数组。本系统将在程序启动时生成出所有棋子的走法预生成数组，在博弈搜索时只做查找和简单的判断操作（下面讨论）。

2.2.3 合理走法的产生

由上面的介绍知道，所有棋子的可能的走法都在系统启动时已经生成。在求某个局面所有可能走法时只需要在走法预生成数组中查找，再根据具体的局面判断是否合理即可。下面用伪代码描述这一过程：

短程棋子以马为例：

```
// PreGen 为全局的 PreGenStruct 结构变量
void Gen_KnightMove(PositionStruct p, int SrcSq)//SrcSq 表示棋子所在的位置
{    int i = 0;
     int DstSq,  LegSq;
     DstSq=PreGen.ucsqKnightMoves[SrcSq][i]
     while(DstSq != 0)  //对马在该位置的每个可能的走法
     {   LegSq=PreGen.ucsqKnightPins[SrcSq][i];//取得该位置对应的马腿位置
         //只需要判断可达的位置没有己方的棋子且马腿位置没有棋子即可
         if(p.ucpcSquares[DstSq] is not self-piece && p.ucpcSquares[LegSq] have no piece)
             AddMove;  //把走法加入到走法列表中
         i++;
         DstSq = PreGen.ucsqHorseMoves[SrcSq][i];
     }
}//其他短程棋子与此类似
```

远程棋子的走法与此类似。从上面可以看出，通过走法预生成数组直接来取走法，然后根据具体局面作少量的判断即可获得棋子全部合理的走法，这会在搜索过程大量调用，试验证明也是有效的（实验结果参看表 6.1）。

2.2.4 人类走法合理性判断

人类走法合理性判断是为了防止人类不按规则走棋。可以用如下方法：首先根据具体局面和走法预生成数组确定用户所动棋子所有合理的走法（集合），然后看

中国知网 https://www.cnki.net

用户所下的走法是否在合理的走法集合中，在合理的走法集合则通过，否则它提醒用户从新下棋。

2.3 小结

上面讨论了中国象棋棋盘表示的几种形式：棋盘数组，棋子数组，棋盘-棋子联系数组，扩展的棋盘-棋子联系数组以及带位行位列信息的扩展棋盘。它们的形式越来越复杂，占用内存越来越多，但也带来操作，运算上的高效率，突出表现在：棋盘-棋子联系数组比棋盘数组在遍历某一方所有棋子时快（由 90 次降为 16 次），同时也是其改进型棋盘的基础；扩展的棋盘-棋子联系数组在棋子行与列的计算、判断棋子是否走出界、棋子是否已过河、走步是否过河、棋子颜色等的判断都有非常高的效率；带位行位列信息的扩展棋盘可以获得车炮的着法预生成数组，以便高效的获得车炮的走法；这些操作和运算在搜索过程中会被频繁的调用，因此使用带位行位列信息的扩展棋盘应会有较快的运算速度。

在走法生成中讨论了如何生成中国象棋所有棋子的走法预生成数组，把所有棋子在棋盘上任意位置上的最大可走步预先求出来保存在内存中。在后面的搜索过程中直接取出来，再根据当前具体局面做简单判断就可以求得棋子合理的走法，这比每次搜索时都重新计算棋子最多可达的位置然后再判断是否合理要快得多。这是典型的以空间换时间的策略。根据 PreGenStruct 结构，需要大约 102KB 的内存消耗，在计算机存储能力大幅度提高，一般 pc 机都在 512MB 时，这是完全可以接受的！

在实验中，使用带位行位列信息的扩展棋盘加上走法预生成数组比使用10×9棋盘在走法产生上提高了 50%的效率（实验结果可以参看表 6.1）。

中国知网 https://www.cnki.net

第三章 局面估值

局面估值就是通过既有的棋类知识来评估一个局面优劣的过程。从象棋的棋力上考虑，在搜索之外，局面估值是最重要的部分，因为对实际局面的评价的好坏，影响着今后局势发展的趋势。这一过程对具体的棋类知识依赖程度很深，但是仍有一般性的规律可循。目前在计算机象棋博弈中常用的估值方法主要采用静态估值方法[6]。同时由于随着搜索层数的加深，叶子节点的数目迅速上升，估值函数被数以百万次的调用，花费大量的运算时间，如何提高估值函数的速度，也成了博弈性能改进的重要话题。下面分别从这两个方面及其关系介绍局面估值。

3.1 估值函数(Evaluation Function)

本系统的估值函数包括:棋子固定价值，棋子位置附加值，棋子灵活性，棋子对棋盘的控制，棋子间的关系几部分。

3.1.1 棋子固定价值

棋子固定价值指的是对棋盘中的每一个棋子，根据它的重要性和走法的丰富程度赋予其一个特定的值。在棋盘上，棋子多者占优，棋子少者为劣。根据经验[5]，可以让一个车价值为500，一个马价值为300，一个兵价值为100等等。在中国象棋的对弈中， 由于一般以将死对方的将(帅)作为结束，因此，通常赋值的规则是为将(帅)赋予一个远大于其他棋子的子力之和的值。

但由于棋手选择的策略和习惯不同，对每个棋子的认识也不同，对棋子的赋值也有一定的差异，同样是一个卒，在高手眼里，所能够起的作用可能不次于马、炮。因此对子力的评估还要充分考虑算法和程序的特点。本系统暂不考虑此情况。

可以用下面的表达式求某一方的棋子固定价值：

SideValue = sum(PieceNum*PieceValue);

其中PieceNum是某种棋子的数量，PieceValue是该种棋子的价值，sum是对各种棋子的总价值求和。 如果红色的棋子价值总和大于黑色的棋子价值总和，通常这意味着红方的局势优于黑方。而红黑双方的SideValue之差越大，红方的优势也就越大。

中国知网 https://www.cnki.net

3.1.2 棋子位置附加值

同样的棋子在不同位置上起的作用是不同的，最明显的是兵(卒)过河之前与之后它的作用和攻击力以及对对方的威胁显然是不一样的，而当兵卒一旦到了底线附近，它对将(帅)的威胁度将大大下降。可以用下面的常数组表示红兵在不同位置的附加值[5]（黑棋在上）：

```
const int BA0[10][9]=
{
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
    {90,  90,  110,  120,  120,  120,  110,  90,  90},
    {90,  90,  110,  120,  120,  120,  110,  90,  90},
    {70,  90,  110,  110,  110,  110,  110,  90,  70},
    {70,  70,  70,   70,   70,  70,   70,  70,   70},
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
    {0,   0,   0,   0,   0,   0,   0,   0,   0},
}。
```

3.1.3 棋子灵活性

棋子灵活性描述的是棋子的活动范围，即棋子向各处调动的可能性大小。棋子的威力能否充分发挥，与灵活性直接相关。本方棋子可以走的点越多，说明本方棋子的灵活性越大。例如车在纵横线上遇到障碍物、马被周围棋子绊脚等，都降低了灵活性，也降低了威力的发挥，灵活性的计算是把棋子在棋盘上所能够移动到达的位置数作为灵活性计算，给予评分。

可以用下面的表达式求某一方棋子灵活性：

Mobility = Sum(MoveNum*MoveValue);

其中，MoveNum是某种棋子的合法走法数量，MoveValue是该种棋子每一走法的价值，Sum是对所有棋子的灵活性价值求和。Mobility就是所有棋子的灵活性分数。

3.1.4 棋子对棋盘的控制

一方对棋盘控制的棋盘区域越多，那么他就越具有主动性。在象棋中，如果一位置落在某方棋子的合法走步上，就可以认为被该方控制。如果某一位置同时落在双方的合法走步上，可以根据双方控制该位置的棋子数量及棋子价值来决定孰优孰劣。能控制更多位置的一方应在这项评分上占优。

中国知网 https://www.cnki.net

3.1.5 棋子间的关系

在中国象棋博弈中，每个棋子都不是孤立存在的，他们之间构成了各种相互关系。如棋子1下一步将被吃掉，则应该给一个负的附加值。而如果周围有己方棋子对其进行了保护，也就是说，对方在理智的情况下不会去吃棋子1，那么棋子1没有受到威胁，价值不变。

棋子关系的评估应考虑到该谁走棋的问题。如果某个红马落在黑炮的合法走步之内，但此时轮到红方走棋，应认为红马受到的威胁较轻。而如果此时轮到黑方走棋，就应认为红马受到的威胁很大，应减去一个相对较大的值了。

3.2 估值的速度与博弈性能

开发人员可以向估值函数加入大量的棋类知识，使之对于局面的评估更为精确。但是，博弈系统的性能，速度，棋类知识一般满足下面的关系[5]：

Performance(性能)=Speed(速度)×Knowledge(知识)

且向估值函数加入越多的棋类知识，估值函数的速度也就越慢，因为所要进行的计算也相应增加。因此过于简单的估值函数和过于复杂的估值函数同样性能不佳。在同等的知识含量下，速度越快，性能越高。在同等速度之下，知识量越大性能越高。在速度和知识量二者的相互作用下，开发者要寻找的是一种平衡，是能够使性能最大化的速度和知识量。

在本系统中，使用终点估值（end-point envaluation），意思是当叶子节点到达时，使用估值函数对一个局面进行评估。这样的方法思路清晰，容易设计，而且模块间独立性高，同搜索算法的耦合度很低。你可以轻易的更换估值函数，只改动极少的代码；你也可以随意使用任何估值方法来评估整个棋盘，最终给出估值，但这种方法的速度较慢[5]。

前面讨论过用Zobrist方法计算哈希值的方法。如果将一个局面的估值Score也看成是一系列棋子价值的总和，那么采用增量式计算的Score就是对这个值的加加减减。与Zobrist哈希方法相似，这一估值方法将棋子在棋盘上不同位置上的价值放进事先建立的一个多维数组T当中，如T[pieceType][square]。该数组是事先确定的，根据棋子在不同位置上的优劣程度，在数组中存放了不同的棋子价值。一些参考资料将这一方法称为棋子价值表（Piece Square Table)估值法。同终点估值法相比使用棋子价值表的过程要快很多。但是，尽管使用棋子价值表估值能获得不可思议的速度，不幸的是，这一方法的估值精度很不理想。原因是棋子价值表无

中国知网 https://www.cnki.net

法刻画出棋子间的关系。由于位置的优劣随着棋子的移动而变化，使用棋子价值表无法知道哪些棋子被对方威胁，哪些棋子受到己方保护。

而对本系统，由于采用了走法预生成。对终点估值的速度有很大的改善。同时限于时间，没有深入探讨如何把Zobrist哈希方法用于博弈系统的估值了。

3.3 估值与走法预生成数组相结合

在对某一局面估值时需要遍历所有棋子，统计与该棋子关联的位置以确定棋子的灵活性、对棋盘的控制以及与其他棋子的关系等！估值的调用在搜索过程频繁调用，在实验中发现，估值是最耗费时间的部分。在10×9棋盘结构不带走法预生成数组时对初始局面估值一百万次要25秒以上，而搜索到7层以上，估值会达到几百万次这，形成了速度的瓶颈！

把走法预生成数组用于统计某一棋子的关联位置，用伪代码描述如下

```
int GetRelatePiece(Position& Pos,int SrcSq)
{   int nPosCount = 0;
    switch(Pos.ucpcSquares[SrcSq] piece type)
  {   case KING_TYPE:
          DstSq = PreGen.ucsqKingMoves[SrcSq][nPosCount];
          while(DstSq != 0)
          {    AddPoint; //把该位置加入关联位置
               nPosCount++;
               DstSq = PreGen.ucsqKingMoves[SrcSq][nPosCount];
          }
          break;
        ……//其它棋子的关联位置如法炮制
    }
}
```

可以看出求关联位置与求走法产生类似。只是条件更宽松：只要是可达的位置都是关联位置。

通过与走法预生成数组结合，估值函数的速度提高了38%左右（实验结果可以参看表6.1）。

中国知网 https://www.cnki.net

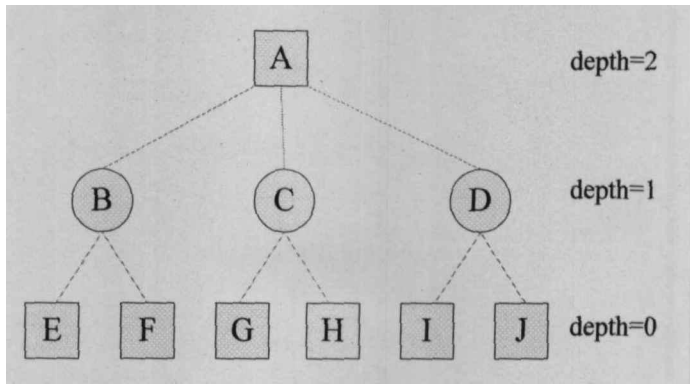
第四章 博弈树搜索技术

在智能过程中，搜索是必不可少的，是人工智能中的一个基本问题——Nilsson 曾这么说过[4]。

博弈事件虽然千差万别，但是在具体博弈事件的背后都有着一些普遍的规律。如跳棋、围棋和象棋等棋类活动中，参加者一般是两人，并且双方的子力在一开始完全相等，局面也是公开的，博弈双方可以清楚的知道对方所采取的行动和目前的局面。这样的博弈事件是博弈中最简单的一种形式，称为:二人零和、全信息和非偶然博弈。中国象棋就属于这样的博弈事件，参与双方在任何时候都完全清楚每一个棋子是否存在，位于何处。本章讨论的博弈树搜索技术就是针对这种博弈事件展开的。

4.1 博弈树（Game Tree）

如果参加博弈的不是一个主体，而是对抗性的敌我双方，则搜索的进程不仅仅取决于其中一方的意愿，同时还取决于对方应付的策略。由此而产生的搜索树，通常称为博弈树[8]。在计算机博弈系统中博弈树可以看成是把计算机和用户所有可能走法和局面罗列出来的一颗树。下图就是典型的博弈树结构。



深度为2的博弈树

图 4.1

如图4.1所示，博弈双方交替地按合理走法把树展开，树的每一个节点都表示某一个特定局面。根节点A表示的是当前需要计算的局面，中间节点B, C,D表示的是对弈过程中可能出现的某个局面，叶子节点E, F……是树的最底端。叶子节点和根节点之间的最大距离，称为搜索深度。整个博弈树描述的是从当前局面出

中国知网 https://www.cnki.net

发，包含所有可能的对弈过程的搜索树。

博弈程序的任务就是对博弈树进行搜索，找出当前最优的方案。Neill Graham形容此过程类似于在一个状态图中寻求从初始状态通向终了状态的过程[9]。只是状态图搜索仅有一个主体参加，仅是单方面做出的路径选择。而博弈树的搜索则有对立的双方参加，一方只能做出一半选择，而这一半选择的目的是使对方远离其竭力靠近的目标。也就是说状态图搜索是纯粹的或树（OR tree），而博弈树搜索是与或树（AND/OR tree）。

如果计算机能从初始局面开始，建立一棵博弈树，树的所有叶子节点都是分出胜负的局面，那么这棵树称为完全博弈树。如果可以建立完全博弈树，中国象棋的问题就可以得到解决[5]。也就是说电脑总能找到一个取胜的路径：象棋的不败算法。但是中国象棋其每一局面可有约20~60种走法。以平均40种走法计，建立一棵（双方各走50步）搜索树就需生成40^{100}约10^{160}个节点。这远远超出了当今计算机的处理能力。即使生成一个节点仅需10^{-8}秒（实际一般的PC机目前为：10^{-7}秒以上），生成这棵树也要10^{140}年以上[5]。显然这是不切实际的；也就是说，建立这个博弈树已经远远超出了当代计算机的处理能力，必须得有其他切合实际的方法。

解决方法就是在人类对手可以接受的时间内，让博弈树扩展到尽可能的深度，然后对没有分出胜负的叶子节点进行评估，以找到对自己尽量有利的走法。为了找到对自己尽量有利的走法，就要求搜索的深度尽可能深；而搜索深度越深所需时间也越长，这就要求程序在满足性能的同时要使搜索效率尽量高。

下面介绍搜索算法及如何利用各种策略来改善搜索算法以提高搜索效率。

4.2 极小极大值算法（Minimax Algorithm）

博弈树搜索属于对抗性搜索（Adversarial Search）。在博弈过程中，任何一方都希望自己取得胜利。在某一方有多个行动方案可供选择时，他总是挑选对自己最为有利而对对方最为不利的那个行动方案。因此某一方在走棋时必须遵守如下原则：考虑到对自己最不利的情况，从最坏的可能中选择最好的；并假定对手不会犯错误，即对手总是选择对他最有利的步子走，所以不能采取任何冒险行动。这个原则就叫极小极大原则，用此原则指导搜索的算法就叫极小极大值算法[10,11,13]，其基本要点如下[14]：

(1)设博弈的双方中一方为A，另一方为B。极小极大值算法是为其中一方(例

中国知网 https://www.cnki.net

如A)寻找一个最优行动方案的方法。

(2)为了找到当前的最优行动方案，需要对各个方案可能产生的后果进行比较。即要考虑每一方案实施后对方可能采取的所有行动，并计算可能的得分。

(3)为计算得分，需要根据问题的特性信息定义一个估值函数，用来估算当前博弈树端节点的得分（在上一章已介绍）。

(4)当端节点的估值计算出来后，再推算父节点的得分。推算的方法是:对“或”节点，选其子节点中一个最大的得分作为父节点的得分，这是为了使自己在可供选择的方案中选一个对自己最有利的方案;对“与”节点，选其子节点中一个最小的得分作为父节点的得分，这是为了立足于最坏的情况。这样计算出的父节点的得分称为倒推值。

(5)如果一个行动方案能获得最大的倒推值，则假定它就是当前最好的行动方案。

以下图为例描述这一过程：

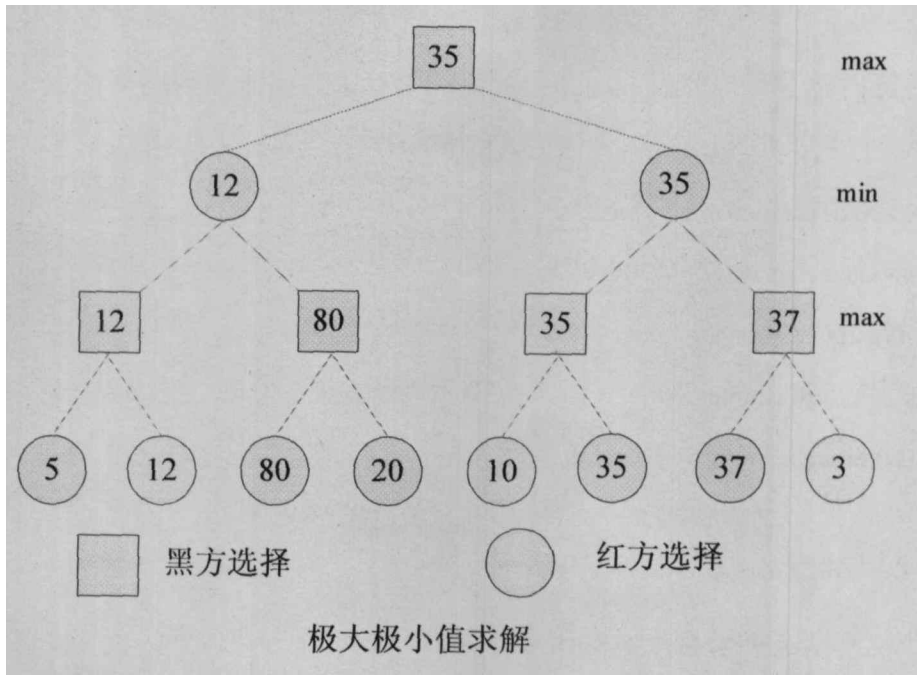


图 4.2

在图4.2中（当前搜索深度为3层），先对最底层（第0层）下对各个可能出现的棋面进行估值，按照由底向上推。首先第1层黑方各节点从其诸子节点（第0层）中选择最大的值，然后第2层红方各节点从其诸子节点（第1层）中选择最小的值，依次交替类推，推出最上层的节点值，从而选择出博弈路径（使根节点获得最大值的那一枝）。

中国知网 https://www.cnki.net

在使用极小极大值算法进行搜索时要考虑是使用广度优先（Breadth First Search）还是深度优先（Deep First Search），广度优先一般需要使用大量的内存来存放生成的局面。以中国象棋为例，并假定用90个字节的数组来表示局面（本系统使用的带位行位列的扩展棋盘为372字节），当搜索深度为8层时内存大于10^{14}Byte，即大约需要10^5 GByte，这是目前计算机配置不可能达到的。并且当搜索深度加深，内存需求呈级数式增长。故只能选择深度优先搜索。深度优先搜索任何时候只保存与其层数相等的节点个数。

可能是冯·诺依曼（John Von Neumann）最早提出极小极大值算法，冯氏于1928年提出"二人零和博弈的极小极大解决方法（The minmax solution for a two-person zero-sun game）[5]"；而后的大约50年里极小极大值算法没有什么变化和改进，直到Knuth和Moore在1975年提出了负极大值（Negamax）方法[15],这个方法可以兼顾双方的立场，消除与节点和或节点的差别，以统一的方式处理。

负极大值（Negamax）方法原理是：对于没有后代的节点和搜索层次达到极限的节点，仍采用静态估值函数来计算它们的估计值,但要站在该节点走棋方的立场上来估值；对于其他节点，均令父节点的估计值为诸子节点的估计值的负数的极大值[4]。它是极小极大算法一种更好的表示形式。用伪代码描述如下：

```
int NegaMax(position p, int d)//p表示局面 ,d为搜索深度
{   int value, bestvalue = -INFINITY;
    if (game over or depth <= 0) //游戏结束或搜索深度达到
        return winning score or eval(); //返回表示输赢的值或返回端节点估值
    GenMove(); //产生局面所有可能的走法
    for(each possible move m) //对每一种可能的走法m
    {   MakeMove(m); //采取这个行动
        value = -NegaMax (p, d-1); //递归调用Negamax向下搜索节点
        UnMakeMove(m) ; //恢复原来的局面（取消前面采取的行动）
        if(value > bestvalue)
            bestvalue = value; //取最大值
    }
    return bestvalue; //返回最大值
}
```

在此算法中，对每个节点的估计值都是站在各自的立场上估值，故不需要区

中国知网 https://www.cnki.net

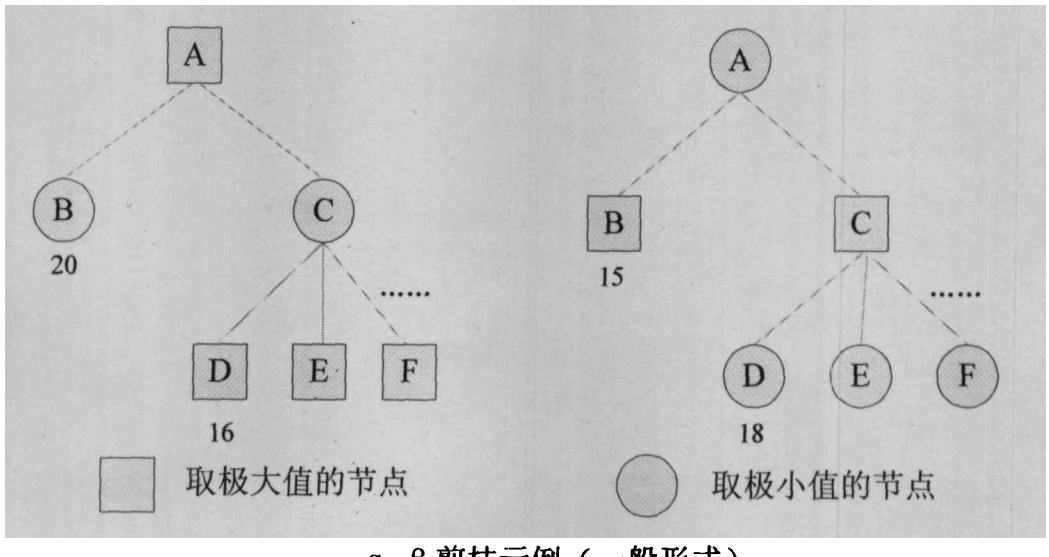
分与节点和或节点的区别；父节点取诸子节点中估值最小的负值（负极大值）。这样可以消除与节点和或节点的差别，使代码简洁优雅。在下面的伪代码中将用负极大值形式来描述。

4.3 $\alpha-\beta$ 剪枝算法及其改进策略

Bruno在1963年首先提出了 $\alpha-\beta$ 算法[5]，1975年Knuth和Moore给出了 $\alpha-\beta$ 算法的数学正确性证明[15]。可以说 $\alpha-\beta$ 剪枝算法是博弈树搜索的基础技术，是博弈搜索算法到目前为止使用最广泛的算法。但 $\alpha-\beta$ 剪枝也有其弱点，可以通过一些策略来改进、增强 $\alpha-\beta$ 剪枝算法。下面讨论 $\alpha-\beta$ 剪枝算法及其改进策略。

4.3.1 $\alpha-\beta$ 剪枝算法(Alpha-Beta Pruning Algorithm)

使用负极大值算法进行搜索时，随着搜索深度的增加，虽然可以得到“最优解”，但是花费的时间也大大增加。之所以会发生这样的情况，是因为算法每次都会去分析全部的节点，对中国象棋而言，每一局面可有约20~60种走法。以平均40种走法计，双方各走2步搜索树就需生成40^4个节点，在实验中也发现随着深度的增加节点与时间呈指数式增长；而人类根据经验不会去分析绝大多数的节点。例如，很少有人会在开始的几步内去移动“将”，“士”等棋子。实际上，在对博弈树的搜索过程中，许多节点的估值的大小并不影响其根节点的值[15]。如下图：



α - β 剪枝示例（一般形式）

图 4.3

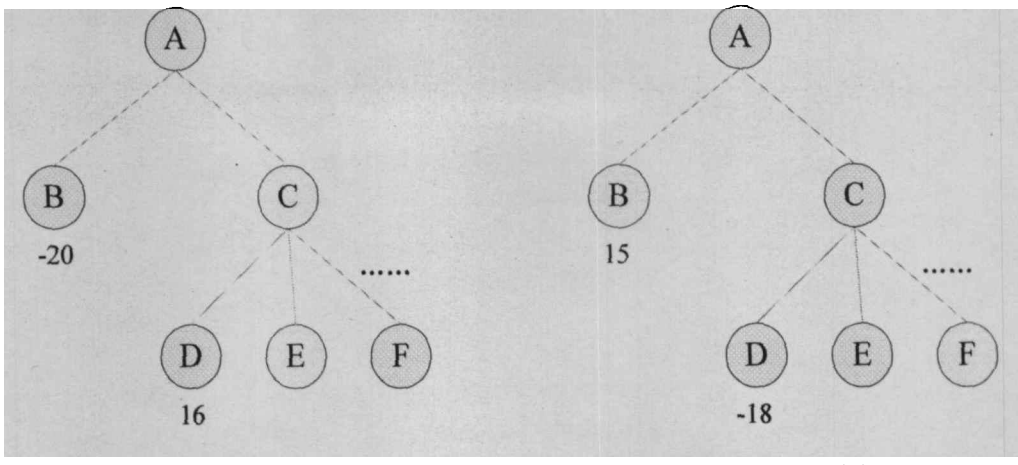
如图4.3所示，在左半边所示的一棵极大极小树的片断，节点下面的数字为该节点的估计值，节点B的值为20，节点D的值为16,由此可以确定C的值一定不会

中国知网 https://www.cnki.net

超过16(因为C=min(D,E,F，...))，从而可以推出A的值一定不会小于20(因为A=max(B,C))，所以分析了D节点以后，E，F,...节点就不再需要分析了，它们对搜索不会做任何贡献：极大值冗余。这种在分析了D后，减去其后继兄弟节点的情况，被称为α剪枝(alpha cutoff)。α剪枝其实是站在己方（取极大值的节点）立场上来实现剪枝：在已知一步棋:B的结果的前提下，当发现走另一步棋:C的潜在危险更大（估值比B走法的估值更小）时，停止试探（搜索）。

在右半边所示的一棵极大极小树的片断描述的情况形成对偶：极小值冗余。节点B的估计值为15，节点D的估计值为18，则节点C取的值一定不会小于18(因为C=max(D,E,F，...))，从而可以推断出A的值一定不会大于15(因为A=min(B,C)),所以当分析了D节点以后，也不用再分析E，F，...节点了。这种在分析了D后，减去其后继兄弟节点的情况，被称为β剪枝（beta cutoff）。β剪枝其实是站在对方（取极小值的节点）立场上来实现剪枝：在已知一步棋:B的结果的前提下，当发现走另一步棋:C的潜在危险更大（估值比B走法的估值更大）时，停止试探（搜索）。把这两种剪枝策略加入极小极大算法就形成$\alpha-\beta$剪枝算法。

由于估值的时候只站在某一方的立场上，所以在剪枝的时候要区分极大值节点与极小值节点。用负极大值形式则不需要区分。如下图4.4：



α 剪枝　　β 剪枝

α - β 剪枝负极大值形式

图 4.4

图4.4与图4.3表示的信息完全一样，只是估值时分别站在走棋方的立场上。并假定估值函数是一样的：对同一局面，一方估值为value，则另一方对该局面估值为-value。用负极大值方法分析：对左半边图，节点C的值最小为-16（因为C=max（-D,-E,-F，...）），节点A的最小值为20（因为A=max（-B,-C）），而C返回给

中国知网 https://www.cnki.net

A的负极大值不会超过16，所以当分析了D节点以后，不用再分析E，F，...节点了；对右半边图，节点C的值最小为18（因为C=max（-D,-E,-F，...）），节点A的最小值为-15（因为A=max（-B,-C）），而C返回给A的负极大值不会超过-18，所以当分析了D节点以后，不用再分析E，F，...节点了。可见此时不需要区分极大值节点与极小值节点，对所有节点同等对待：只需用令父节点的估计值为诸子节点的估计值的负数的极大值即可。用伪代码描述$\alpha-\beta$剪枝（负极大值形式）如下：

```
int AlphaBeta(int depth, int alpha, int beta) //*说明 1
{   if (game over or depth <= 0) //游戏结束或搜索深度达到
        return winning score or eval(); //返回表示输赢的值或返回端节点估值
    GenMove(); //产生局面所有可能的走法
    while (MovesLeft()) //对所有的走法
    {   MakeNextMove();
        val = -AlphaBeta(depth - 1, -beta, -alpha); //递归调用
        UnmakeMove();
        if (val > alpha) //找到一步更好的棋
            alpha = val;
        if (alpha >= beta) //*说明 2
            break; //剪枝;
    }
    return alpha;//返回极大值
}
```

*说明 1 与 2：对$\alpha-\beta$剪枝负极大值形式（站在走棋方各自的立场估值）：alpha 表示当前节点估计值的下限（已经搜索到子节点中最好的），beta 表示父节点允许当前节点估计值可以到达的上限，超过此上限，表示父节点有更好的走法（已经存在，可以参考示例），不会走此步棋，故引发剪枝。

实验证明当搜索深度大于四层时，$\alpha-\beta$剪枝算法搜索的节点数与花费的时间比单纯的极小极大值算法降低一个数量级还多。但搜索到六层已经让人不可容忍了：接近 500 秒（实验结果请参看表 4.1 与表 4.2）。$\alpha-\beta$剪枝算法还有待改进。例如下图 4.5 所示的情形$\alpha-\beta$剪枝就无计可施了。

中国知网 https://www.cnki.net

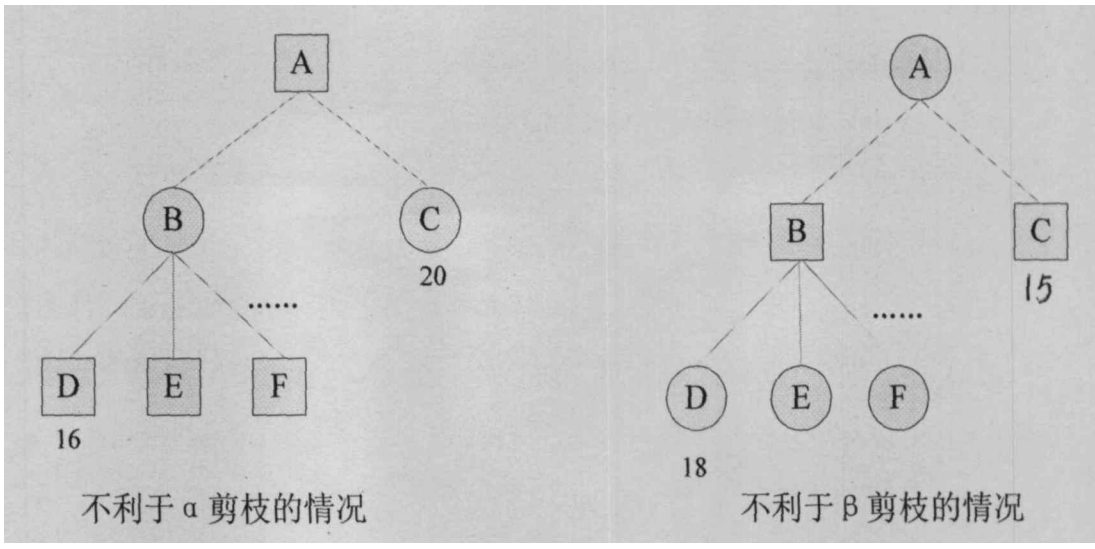


图 4.5

可见$\alpha-\beta$剪枝算法依赖于着法的寻找顺序。如果你总是先去搜索最坏的着法，那么就不会引发剪枝，因此该算法就如同极小极大算法一样，效率非常低。该算法最终会找遍整个博弈树。

针对$\alpha-\beta$搜索的特点，有许多种增强策略被提出，这些策略有些已经被证明为是非常有效的，下面是$\alpha-\beta$的增强策略的介绍。

4.3.2 窗口原则(Window principle)

在$\alpha-\beta$剪枝过程中，初始的的搜索窗口往往是从-8（即初始的 alpha 值）到+8（初始的 beta 值），在搜索进行中再不断缩小窗口，加大剪枝效果，这种估计是可靠的，但却不是高效的。如果一开始就使用一个较小的窗口，那么就有可能提高剪枝效率，这就是窗口原则也叫有限窗口法[4]。

定理：

设博弈树中任意节点 P，P 的极小极大值为 f(P), f(P)的近似估计值为 V，误差为 e,如果把[V-e, V+e]作为窗口执行$\alpha-\beta$算法后有三种情况：

1) 若 MM <= V-e, 则 f(P) <= V-e;

2) 若 MM >= V+e, 则 f(P) >= V+e;

3) 若 V-e < MM < V+e, 则 f(P)=MM;

其中 MM 是以[V-e, V+e]为窗口执行$\alpha-\beta$算法获得的值。

此定理与在陆汝钤老师的书[4] Page391 上描述有差异。

上面定理中的情况 1）和情况 2）分别称为偏高和偏低。这两种情况虽然没有准确的求出 f(P)的值，但却对 f(P)的位置提供了有用的信息，可以以此为基础实行进一步的搜索。

中国知网 https://www.cnki.net

使用窗口原则的算法主要有：

1）FAlphabeta 算法(Fail-soft-alphabeta 算法)

其基本思想[4]：先定义一个窗口，如果极小极大值正好在这个窗口内，算出即可。否则，得到有关极小极大值的位置信息（探路），为下一步提供方便

AlphaBeta 算法与 FAlphabeta 算法在定理所示的第 3）种情况下运算完全一样，情况 1）和情况 2）则可以提供一个参考值。在下面介绍的渴望搜索将使用该参考值。

2）渴望搜索（Aspiration Search）[20]

渴望搜索是建立在 FAlphabeta 算法基础之上的，第一步先设定窗口[alpha, beta]，第二步用 FAlphabeta 算法进行探路，成功（若返回的值在窗口内表示成功）则可，不成功则修改窗口继续执行第二步。修改窗口方法如下：若 FAlphabeta 算法返回的值小于等于 alpha，即偏高，则使 alpha 值减小；若返回的值大于等于 beta，即偏低，则使 beta 值增大。

对于渴望搜索的效率，各资料的上反映不一[5]。一些评论者认为，由于渴望搜索的窄窗引发更多的剪枝，可以大幅度提高效率。同时另一些研究者如 J.Schaeffer在其论文[16]中指出渴望搜索只在深度较深的时候略优于$\alpha-\beta$剪枝算法。可以看出，在使用时，由于环境不同效率有很大差异。

3）极小窗口搜索（Minimal Window Search/PVS）

极小窗口搜索，也被称为是主变量导向搜索（Principal Variation Search），常常简称为PVS算法或者NegaScout算法[17]（在形式上有一点区别）。这个算法应用非常广泛。其基本思想：预测第一个分枝（主变量）是所有分枝最好的，也就是假定它为最佳走步，对它进行完整窗口[alpha, beta]的搜索并得到一个返回值v，对后面的儿子节点依次用极小窗口（也被称为是零窗口）[v,v+1]来进行搜索:如果搜索返回值大于零窗口，则证明这一分支可能为主变量，对它进行窗口为[v+1, beta]的搜索；如果返回值小于零窗口，这一分支就可以忽略，因为它的最佳走步还不如已有的走步。可以看出与$\alpha-\beta$搜索算法相比增加的时间消耗是对零窗口[v, v+1]搜索的调用，如果零窗口[v, v+1]调用返回的值比零窗口小，那么就可以节约一个完整窗口[alpha, beta]搜索所花费的时间，并且大于零窗口后的再搜索相对于完整窗口也缩小了范围，所以其搜索产生的博弈树更小。其伪代码如下：

```
int PrincipalVariation(int depth, int alpha, int beta)
{    int best，score;
```


中国知网 https://www.cnki.net

```
if (game over or depth <= 0) //游戏结束或搜索深度达到
    return winning score or eval();  //返回表示输赢的值或返回端节点估值
GenMove(); //产生局面所有可能的走法
MakeMove(m) ; //产生第一个节点
best = -PrincipalVariation(depth -1, -beta, -alpha) ; //全窗口搜索第一个节点
UnMakeMove(m); //撤销第一个节点
for(each possible move m) //每一种可能的走法(从第二个节点起)
{   if(best < beta)//不能剪枝
    {   if(best > alpha)
            alpha = best;//缩小搜索窗口
        MakeMove(m) ;//创建一个子节点
        score = -PrincipalVariation(depth -1, - alpha-1, -alpha)//空窗探测
        if(score>alpha && score < beta)//探测结果偏高(比前一枝好)但没引发剪枝
            best = -PrincipalVariation(depth -1, -beta, -score) ;//缩小窗口
        else if(score > best)// 比前一枝好但偏低或引发剪枝
            best = score;
        //其他比前一枝差的情况不用再考虑了
        UnMakeMove(m) ;//撤销创建的子节点
    }
}
return best;//返回最佳值
}
```

试验证明极小窗口搜索策略可以成倍（>4层以后）的提高$\alpha-\beta$搜索算法的效率。且效果随搜索深度的增加越来越好（实验结果请参看表4.1与表4.2）。如果能在很早的时候就发现最好的那一枝，那么余下的搜索都可以用空窗检测并且不会引起偏高。这将极大的提高搜索效率。

前面介绍了三种使用窗口原则的$\alpha-\beta$搜索算法，他们可以提高$\alpha-\beta$搜索算法的效率，但单纯使用这些算法还不能达到实用的水平。它需要和下面的策略结合使用。

4.3.3 置换表(Transposition Table)与哈希表（Hash Table）

在搜索过程中，经常会在相同的或者不同的路径上遇到相同的局面，即着法

中国知网 https://www.cnki.net

顺序的不同可能会导致相同的局面。那么能否把已经搜索过节点的结果保存起来，下次搜索到相同局面直接取值而不进行再次搜索呢？想法是可行的。这种方法就叫置换表[5]。

置换表基本思想：在搜索进行中，用一张表把搜索过的节点结果（包括搜索深度，估值类型：准确还是上下边界）记录下来，在后继的搜索过程中，查看表中记录，如果搜索的节点已经有记录（子树的深度大于或者等于当前的新节点要求的搜索深度），它的信息就可以直接利用，这样可以避免重复搜索很多子树。置换表是一种内存增强技术，以空间换时间。把置换表用于$\alpha-\beta$剪枝算法的伪代码如下：

```
int AlphaBeta_TT(int depth, int alpha, int beta)
{   value = LookupTT(depth, Index);//查询置换表
    if(value is valid)//察看返回的值是否合理:有记录且depth小于等于已有记录搜索深度
        return value; //合理返回记录的值
    search with AlphaBeta_TT//没找到，进行α－β搜索过程，求出当前局面的值 value
    StoreToTT(depth, value, Index)//将搜索到的当前局面估值保存到置换表
    return value;
 }
```

从上面伪代码可以看出，该技术的核心是如何快速的保存与查找置换表并使得置换表的空间使用在可以接受的范围之内，二者缺一不可。因此置换表最好设置成随机存取类型并且可以在常数时间内插入，当然要在当前可以接受的存储空间内进行。答案是哈希表！

定义一个大小可以接受的数组，让每一局面在数组中对应唯一的位置（这样数据的取出与插入可以实现随机读写），但并不保证每一个位置对应唯一的局面（在2.2节中已经讨论把所有局面保存起来是不可能的）。不同局面对应数组中的同一存储单元的情形叫冲突（数组越大冲突越小），但对一次搜索而言，冲突发生的概率并不大[5]；即使在某个搜索过的局面在该数组中没有找到，这也只是回到为改进以前：对它进行于$\alpha-\beta$搜索，因此使用哈希表来实现置换表是可行而有价值的。可以用下面的数组来描述哈希表[5]：

```
struct HASHITEM     //24字节
{   int64 checksum;//64位哈希值用于验证表中的局面是否为要找的局面
    int depth;//该局面求值时的搜索深度
```


中国知网 https://www.cnki.net

```
    enum{exact, lower_bound, upper_bound}entry_type;//所求值类型
    double eval; //所代表局面的估计值
}hashTable[HASH_TABLE_SIZE]; //HASH_TABLE_SIZE为哈希表的大小
```

这样，对每一局面计算出一个哈希值（通常是一个 32 位的数）hashIndex，以确定它在哈希表中的位置；计算另一个哈希值（通常是一个 64 位的数）Checksum，以确定它是否为要找的棋局，这个哈希值（64 位的数）在存入的时候已经保存在数组的数据中。当然这个 64 位的数在理论上还是存在冲突的，但发生的机率已经非常小了，几乎可以忽略了。使用时，先查看哈希表 hashTable[hashIndex], 若 hashTable[hashIndex].checksum == Checksum && hashTable[hashIndex].depth 大于等于当前局面需要搜索的深度，则返回 hashTable[hashIndex].eval 作为当前局面的估值。

最后一个问题是如何快速产生哈希值。对每一节点要产生一个 32 位的索引用来定位节点在哈希表中的位置，还要产生 一个 64 位的验证值来验证表中记录的局面是否与当前局面一致。由于计算这两个值得调用十分频繁（每一局面都需要），所以要求求哈希值的过程尽量快。在博弈程序中最广泛使用的快速求哈希值的方法是 Zobrist 于 1970 年提出的[18]。

其方法如下：在程序启动时，首先建立一个多维数组 Z[pieceType][piecePosition]。其中 pieceType 为棋子种类，piecePosition 为棋子位置。本系统中分别取 16 与 256。然后用随机数填充此数组，要求某一局面的哈希值，则将棋盘上所有棋子在数组 Z 中对应的随机数相加，即可得到。为了避免对每一局面都要进行棋盘上所有棋子在数组 Z 中对应的随机数相加（亦可用按位异或操作，本系统使用按位异或操作），Zobrist 方法使用增量式计算方法：在程序的根部（棋局开始时），作一次加总操作求出根节点的哈希值，当搜索一个新节点时只要棋子在移动前将对应的随机数从哈希值中减去，再加上该棋子在移动后对应的随机数即算出了该子节点的哈希值。当然，如果移动的棋子吃了别的棋子，还要减去被吃掉的棋子被吃前所对应的随机数。当对这个局面搜索完成后，再将搜索前减掉的值加上，搜索前加上的值减掉，就恢复了当前节点的哈希值。

实验证明，由于带置换表的$\alpha-\beta$剪枝算法的操作对每一节点都要花费一定的时间进行查找，所以在较浅的搜索当中（前3层），由于命中率较低，搜索节点不相上下，但时间反而有增加的趋势，置换表没有显示出优势；但随着搜索深度的增加，置换表的命中率增加，搜索的节点数和花费的时间明显加少，当搜索

中国知网 https://www.cnki.net

深度达到5层时都降为$\alpha-\beta$剪枝算法的一半左右（实验结果请参看表4.1）。

MTD(f)算法[22,23]就是一种使用了置换表技术和极小窗口的$\alpha-\beta$剪枝算法。其全称是 Memory –enhanced Test Driver with f and n。其基本思想：在算法之初，要给定一个初始猜测值 f，一个上界和一个下界。初始值越靠近最后要找到的最优走步值效率越高。算法实际上就是不断的应用空窗口的$\alpha-\beta$搜索，缩小上界和下界，并移动初始值使其接近最优走步，当下边界大于或等于上边界，搜索完成。其伪代码如下：

```
int MTDF(int test, int depth)
{   int score, beta, upperbound, lowerbound;
    score = test; //初始猜测值
    upperbound = +INFINITY;
    lowerbound = -INFINITY;
    do { beta = (score == lowerbound ? score + 1 : score);
         score = alphabeta(depth, beta - 1, beta); //空窗检测，其中使用置换表
         (score < beta ? upperbound : lowerbound) = score;
       } while (lowerbound < upperbound); //下边界不小于上边界
    return score; //返回结果
}
```

此算法有很多改进之处，麻省理工学院的 Cilkchess（当今知名国际象棋）就使用了并行的 MTD(f)算法[24]。但在实验中使用单 CPU，发现它不如基于 NegaScout 的综合算法，故没有在下文分析（实验结果请参看表 4.1 和表 6.2）。

4.3.4 历史启发（History Heuristic）

在 2.3.1 节的末尾谈到，$\alpha-\beta$剪枝算法依赖于着法的寻找顺序。如果你总是先去搜索最坏的着法，那么就不会引发剪枝，因此该算法就如同极小极大算法一样，最终会找遍整个博弈树，效率非常低。

Knuth 和 Moore 所做的研究表明[15]，在最理想的情形下建立的博弈树的节点个数为：W^d；其中 W 是博弈树的分枝因子，d 是最大搜索深度。这大约是极小极大搜索建立的节点数的平方根的二倍。也就是说对一棵极大极小树来说，如果极大极小搜索需遍历 10^6 个节点求得结果，那么处于理想状态的$\alpha-\beta$剪枝算法仅需遍历约 2000 个节点就可求得结果。因此如何调整待展开的走法序列，是提高搜索效率的关键。

中国知网 https://www.cnki.net

历史启发就是为了迎合$\alpha-\beta$剪枝算法对节点排列顺序敏感的特点来提高剪枝效率的，即根据历史走法对当前搜索的节点集进行排序，从而优先搜索好的走法。历史启发的思想是：搜索树中某个结点上的一个好的着法，对于其他结点可能也是好的。J.Schaeffer提出所谓好的走法即为可以引发剪枝的走法或者是其兄弟节点中最好的走法[16]。

有两个问题有待解决，1.如何给定历史得分：对于这个问题有两种考虑，一是认为某分支的搜索层数越深，搜索得到的值也就越可靠；二是认为离根节点越近,某一走法所对应的诸局面相似程度越高，越往下分枝越多，相似程度越小。综合以上考虑，Schaeffer 指出每发现一个好的走法就给其历史得分增加 2^{depth} 是适合的。2.如何将一个走法映射到历史得分的数组中：用一个二维的数组，数组的前一个下标表示初始位置，后一个下标表示目标位置。在本系统中使用 256×256 的二位数组。

Schaeffer同时也指出，历史启发对$\alpha-\beta$搜索的增强，对于具体的棋类知识几乎无任何要求。任何棋类的类似搜索过程，只要能定义出合适的走法映射和增量因子就可以轻易的加入历史启发的增强。

实验证明，加入历史启发的$\alpha-\beta$搜索算法，在遍历的节点数目上和时间花费上均远远少于单纯的$\alpha-\beta$搜索算法，而且这一差距随搜索深度的增加而增加。仅仅在最大搜索深度为4层的时候，二者在节点数目和搜索时间上的差距都接近10倍了（实验结果请参看表4.1与表4.2）。这一调整节点排列顺序手段所带来的效率提高充分说明了对于$\alpha-\beta$搜索，节点的排列顺序是极为关键的问题。

4.3.5 迭代深化（Iterative Deepening）

迭代深化是一个不断求精的过程，是先进行深度为1的搜索，再进行深度为2的搜索，如此类推，先N层搜索，然后N+1层搜索，直到搜索分配的时间耗尽。对博弈树进行多次遍历，并不断加深其深度，用于控制搜索的时间。在象棋博弈当中，开局、中盘和残局时的分枝因子差别很大，因而导致了同一深度的搜索在不同的阶段时间花费上差异很大。在中盘可以搜索5层的时间在残局往往可以搜索7~8层，对于棋子很少的残局可以搜索十几层甚至几十层[5]。如果用时间来限制搜索的深度，同人类棋手的习惯更为相近。并且，在决定胜负的残局，可以搜索得更深。

效率问题：迭代深化的过程所带来的额外的时间开销与分枝因子有很大的关系，分枝因子越大额外的时间开销所占的比重越小。对中国象棋而言分枝因子在

中国知网 https://www.cnki.net

40左右，进行$\alpha-\beta$搜索d层大概是d-1层的5到10倍[5]。所以迭代深化所带来的额外时间花费并不会使搜索效率有明显的降低。但对于分支因子很小的博弈项目来说，额外增加的几次浅层搜索花费的时间就相当客观了（实验结果请参看表4.1与表4.2）。

迭代深化除了控制搜索时间外，还可以和窗口原则，置换表，历史启发配合使用具有很好的效果。窗口原则中，如PVS算法：上层迭代得到的最佳走法可以帮助下一层提高剪枝效率；迭代过程中把前面局面的估值存入置换表，走法历史得分存入历史启发表可以提高剪枝效率,在4.4节中将继续讨论。

4.3.6 其他问题

$\alpha-\beta$剪枝固定深度会引起“水平效应”[4]：如果一个搜索程序前进到某个深度时，自以为找到了一个比较有利的局面，但它不知道，若再深入几步可看到真正的结局是不利的，这种看不到危险的毛病称为负水平效应；另一种情况相反，如果搜索程序在极限搜索深度处发现不利局面，于是决定放弃，但不知道，只要再搜索几步就可以出现“柳暗花明又一村”好形势，这种情况称为正水平效应。针对“水平效应”,Claude Shannon等人提出在叶节点的局面变化剧烈时，继续向下搜索一直到局面相对比较平静为止。这种方法也叫静止期搜索（Quiescence Search）[25]。

在象棋中，在搜索树叶子之上的一层结点(即深度为1的结点)，如果不会有任何延伸的话，那么进行吃子将会立刻得到分数上的提升，不管对方是否有保护和能否回吃。因为它走完这一步之后，深度已经变为0，也就是不会继续搜索对方的棋步了，所以对方在“搜索逻辑上”无法回吃。这造成分数起伏很大，并无法准确体现局面的正确评价值。所以应该对深度为0的叶子结点以下所有吃子节点进行延伸。当然，可能不仅仅认为吃子才是剧烈变化的局面，如将军也可以认为是剧烈变化的局面。

把静止期搜索加入原有的搜索过程只需要在搜索过程中将叶子节点处的估值函数换成静止期搜索函数，就可以了。加入静止期搜索应该能提高象棋的棋力。

还有一些改进方法如：轻视因子（Contempt Factor），机器学习(Machine Learning)[26,27]，并行搜索。

极大极小算法有一个基本的原则，就是在搜索的过程中假定对手具有和自己一样的博弈能力，自己看到的局面对方也看到并有相同的评估准则。而实际的博弈过程并非如此，机器的对手不可能和机器有相同的视野。与水平效应相反，当搜索深度相当深的时候，会发生程序为了防止若干步之后的一个凶险而走出一步

中国知网 https://www.cnki.net

在人类棋手看来是臭棋的招数。其实这时人类棋手对可以导致自己取胜的方法懵然无觉。因此根据棋手在博弈过程中的表现来设定一个轻视因子，以使估值或搜索的过程随对手的强弱而改变，能够最快地取得博弈的胜利。

在人类博弈学习的过程中，往往在钻研了一个棋谱之后，可以应用于相似的局面。在博弈程序里，这种能力是具备学习能力的一个基础。如果将一个局面和与其相似的局面看成是一个模式，那么在博弈的过程中识别这种模式，并采用模式库中的对策，就是一个与人类相似的博弈过程。学习的过程是通过对博弈过程的识别统计，自行丰富模式库中的内容，以提高程序的博弈性能。另一些学习程序则着眼于训练更精确的估值函数。采用人工神经网络的方法来训练估值函数。使用神经元来表示棋盘上的各种信息，在同对手进行博弈时，神经网络通过反馈信息来修正估值函数中各种参数的权值。但对不同的对手可能效果不一样。

并行搜索威力的一般要在多个处理器上体现出来，如战胜了卡斯帕罗夫的“深蓝”。但算法也更为复杂。

4.3.7 各种改进策略作用于$\alpha-\beta$剪枝算法的实验数据

说明：下面两个表是在无人干涉的条件下各种搜索引擎与另一搜索引擎（搜索深度为三）在开局十步所获得的。(机器性能：Pentium4 CPU 2.8GH,512MB 内存，数据结构用的是基本的 10*9 的数组)

表 4.1 各种搜索算法在开局十步平均每步所用的时间(ms)

层数 / 算法	1	2	3	4	5	6	7
极小极大值算法	<1	10	556	23 450	750 199	——	——
基本的 a-ß 剪枝算法	<1	6	106	967	10 984	945 872	2 982 604
Palphabet 算法(PVS)	<1	6	89	649	5 620	432 011	833 142
a-ß 剪枝+迭代深化(ID)	<1	4	106	968	10 931	497 945	1 223 487
a-ß 剪枝+置换表(TT)	<1	3	92	653	5 735	292 232	1 037 435
a-ß 剪枝+历史启发(HH)	<1	4	27	151	1 090	14 109	92 251
MTD(f)+ ID+HH	<1	<1	23	84	576	5 548	31 864

从表 4.1 中可以看出原始的极小极大值算法搜索到第五层几经不可容忍，每走一步的平均时间超过十二分钟；基本的 α－β 剪枝算法在第五层平均时间接近十一秒，提高六十多倍。各种增强策略都能提高 α－β 剪枝的效率，其中空窗口探测(PVS)从第五层开始能提高一倍的效率；单纯的迭代深化由于在迭代需要耗

中国知网 https://www.cnki.net

费时间，从效率上看提高不大；置换表在前三层没什么表现，这是因为置换表操作也要耗费时间，因当其命中率低时效果不佳，但层数较多命中率高时时优势越来越明显；历史启发是这几种增强策略中最好的，在第五层就能提高十倍以上的效率，这也证实了α－β剪枝对顺序的极度敏感。MTD(f)算法在实验中的前几层稍优于PVS算法，但它层数大于六时很不稳定且本身带置换表，它不如下面的综合算法。

各种搜索算法在开局开局十步平均每步评估的节点数

层数 / 算法	1	2	3	4	5	6	7
极小极大值算法	41	1 194	68 012	2 905 726	91 209 909	——	——
基本的 a-ß 剪枝算法	41	549	13 071	118 901	1 323 381	136 411 349	>1.0e8
Palphabet 算法(PVS)	45	610	10 902	77 980	662 546	64 171 823	94 789 546
a-ß 剪枝+迭代深化(ID)	41	465	12 969	118 527	1 303 405	83 698 737	>1.0e8
a-ß 剪枝+置换表(TT)	41	549	10 225	74 732	640 921	37 526 832	112 944 050
a-ß 剪枝+历史启发(HH)	41	233	3 260	15 20	124 829	1 483 811	9 557 842
MTD(f)+IT+HH	41	208	1 981	6558	50 628	455 136	2 763 380

从评估的节点(剪枝效率)看：原始的极小极大值算法搜索到第五层每走一步平均需估计的节点数接近一亿个；基本的α－β剪枝算法平均需估计一百万多个，剪枝可以极大的减少评估节点。各种增强策略都能减少了α－β剪枝所需估计的节点数，其中空窗口探测(PVS)从第五层开始减少为一半；单纯的迭代深化对剪枝的帮助提高不大；置换表在前三层没什么表现，这是因为置换表操作也要耗费时间，因当其命中率低时效果不佳，但层数较多命中率高时时优势越来越明显；历史启发是这几种增强策略中最好的，在第五层就能提高十倍以上的效率，这也证实了α－β剪枝对顺序的极度敏感。MTD(f)算法在实验中的前几层稍优于 PVS 算法，但它下面的综合算法。

4.4 综合搜索算法及其改进

上边讨论了$\alpha-\beta$剪枝算法及各种其改进策略：缩小窗口范围已达到快速收敛（窗口原则）；记录已搜索节点的估值以避免重复估值（置换表）；根据已搜索节点在搜索过程中的表现来排列节点以提高剪枝效率（历史启发）；利用迭代深化实现时间控制，并为下一轮搜索提供有用的信息；还包括一些其他一些特定

中国知网 https://www.cnki.net

的优化。这些方法，策略都从不同程度上改善了$\alpha-\beta$剪枝算法。现在任何一款商业象棋软件都使用了很多的改进策略，并进行有机结合以提高象棋的搜索速度和棋力。

下面介绍的综合算法（NegScout + TT + HH + ID）将使用上面介绍的窗口原则，置换表，历史启发，迭代深化四种策略（其中置换表的大小为48MB、采用始终覆盖策略、包括叶结点估值；历史启发表使用单层结构、每走一步直接清空历史启发信息；迭代深化是从第一层逐层递增的）。

其伪代码如下：

```
int NegScout_TT_HH(int depth，int alpha，int beta）
{   int score, count, value = -ININITY;
    int a=alpha,b=beta,bestvalue= -1;
    if(GameOver）//已分胜负，返回估值
        return Evaluate（pos）;
    if(LookUpTT（depth，a，b，&score, side））//察看置换表中有无当前节点数据
        return  score; //有，直接返回
    if (Depth <= 0)
    {   score  =  eval(pos);
        Store(depth, a, alpha，beta, side);//将搜索过的结果存入置换表（包含叶结点估值）
        return score;
    }
    count = GenMoves（moves）; //产生所有的走法
    GetHistoryScore(moves, count); //获得历史走法的得分
    Sort（ moves，count）; //按历史得分排列走法顺序
    for（each possibly move m）//对每一步可能的走法 m
    {   make move m; //产生 m 的子节点及其子节点的哈希值
        //递归搜索子节点，对第一个节点是全窗口，其后是空窗探测
        value= -NegScout_TT_HH（depth -1, -b, -a）;
        if(value>a && value<b && m!=0) //探测结果偏高(比前一枝好)但没引发剪枝
        {   a = -NegScout_TT_HH（depth -1, -beta, -value）;
            bestmove = m; //记录最佳走法的位置
        }
```


中国知网 https://www.cnki.net

```
            unmake move m; //撤销子节点并恢复当前节点的哈希值
            if（value > a）//搜索命中
                a = value; //保留最佳值
            if（a >= beta）
            {   bestmove = m; //记录最佳走法的位置
                break; //剪枝
            }
            b = a + 1;//设定新的空窗
        }
        store（depth, a, alpha，beta, side）; //将搜索过的结果存入置换表
        storeHstoryScore(bestmove, depth); //将当前最佳走法汇入历史记录
        return a;
    }
    //迭代加深
    while (i < MaxDepth)
    {   NegScout_TT_HH (depth, alpha, beta)  ;
        if(time > MaxTime) //搜索时间到
            break ;
        i ++ ;
    }
```

实验证明，综合算法如果结合10×9棋盘数据结构搜索到7层需要25秒左右；如果结合带位行位列信息的扩展棋盘数组数据结构搜索到7层则可以控制在10秒以内（具体实验数据参看表6.2和表6.3）。综合算法结合带位行位列信息的扩展棋盘数组数据结构已经比较实用了，但根据这四种策略的特点，还可以进一步改善算法。下面就综合算法的改善进行讨论。

1）动态设置置换表的大小

在均匀程度相同的情形下，越大的哈希表有越好的散列度。哈希表越大，冲突越少，性能越佳。因此，定义一个尽量大的哈希表可在某种程度上提高查找效率，鉴于PC环境上配置的内存越来越大，使用几十兆甚至上百兆的哈希表也有可能。但是，哈希数组的定义不应超过物理内存的大小。一旦使用虚存，哈希表的操作速度将大幅度降低。对于相同搜索深度而言，哈希表的大小与性能的关系并

中国知网 https://www.cnki.net

不是线性的。Jonathan Schaeffer曾指出，一般哈希表的大小每增加一倍，约可使命中率提高7%左右[16]。但当哈希表尺寸大到已使其中的冲突接近零的时候，再增大哈希表的尺寸将没有任何实际意义，除非要进行更深层的搜索。

同时置换表的大小最好是一个 2^n 的常量（n 为整数），因为需要通过“ZobristKey() % TableSize()”这个表达式来定位，而“电脑一做除法就成了傻瓜”，所以把置换表的尺寸设置为 2^n 时，除法就可以由右移指令取代。

鉴于上面两点，可以让用户根据机器配置选择置换表的尺寸，尺寸预先设置好为 $2^n \times 24$。若置换表的尺寸从 256K 个条目开始可以设置为 6M、12M、 24M、48M、 96M、 192M、 384M 等。程序运行时根据用户选择来初始化置换表，这样可以充分利用不同用户的机器性能。

实验表明搜索深度为六层时 6MB 的置换表已经够了，再增加置换表的大小对搜索效率没有帮助；七层时需要 96MB 左右的置换表，层数的增加对置换表的增加要求很快（具体实验数据参看表 6.4 和表 6.5）。

2）置换表的覆盖策略

置换表的覆盖策略通常有两种：深度优先和始终覆盖。也有程序把两种策略结合。

本系统参考了国际象棋程序 Fruit 的做法，尝试采用双层结构的置换表：每个 Hash 位置记录 *2* 个局面。记录一个局面时，找到它对应的 Hash 位置中的 *2* 个局面，不考虑该局面本身的搜索深度，而直接覆盖深度最小的那个局面，若该局面搜索深度比搜索深度大的那个局面也大，则同样覆盖。这种策略的本质就是将深度优先和始终覆盖结合起来——原先深度最小的那个局面是始终覆盖的，而另一个个搜索深度更深的局面则被保留下来。此时可以不清空置换表又能保证置换表中的内容基本是新的。

但在实验中，使用双层结构的置换表与始终覆盖的置换表不相上下，深度优先的置换表则明显不如始终覆盖的置换表（具体实验数据参看表 6.6 和表 6.7）。

3）置换表对叶子节点的处理

综合算法在10×9棋盘数组结构下把叶子节点的估值保存到置换表，但有研究者提出不对叶子节点进行哈希处理，因为叶子节点数量巨大，并且哈希查找和插入也有一定的时间耗费。所以哈希处理可能节约不了多少时间，甚至使搜索效率降低。

这个问题应该根据具体的情况而定。对于估值函数复杂，估值过程时间长的

中国知网 https://www.cnki.net

棋类来说，对叶子节点进行哈希处理对性能的提高是肯定的。而对于象棋，则应视估值函数的运算时间而定。当估值函数预算时间长，此时对叶子节点进行哈希操作能提高搜索速度，而如果估值函数速度很快，这就没有必要对叶子节点进行哈希处理了。这样哈希表中存放的节点会少得多，哈希表的尺寸也可以设计得更小。

在实验时，尝试使置换表不保存叶结点的估值，但实验数据表明，这比用置换表保存叶结点估值要慢（具体实验数据参看表 6.8 和表 6.9）。

4）历史启发表的结构

历史启发表应该用什么样的结构：一种方法是用一个[square][square]数组，第二种是用两个[square][square]数组分别记录红方着法和黑方着法的历史得分，第三种是每一个棋子都对应一个[square][square]数组。

本系统使用第二种结构，即用两个[square][square]数组分别记录红方着法和黑方着法的历史得分。这样既可以提高历史信息的精度又不会操作太慢。

在实验中发现采用第二种结构比采用第一种结构 7 层后效率有所提高：7 层提高 7%，8 层提高了 20%（具体实验数据参看表 6.10 和表 6.11）。

5）历史启发表的清空与修正策略

有研究者提出：如果每次搜索时不清空置换表，那么根结点浅层分值在置换表中已经找到，程序就直接做深层次的搜索，如果历史表是空的，那么历史表的启发效率就非常低了，因此不清空置换表而去清空历史表是不明智的做法，这样会导致“历史表信息丢失”。反过来，不清空历史表，历史表中的数据会日积月累，而大部分数据是早期搜索时留下的，有可能开局时的历史着法信息被保留到了残局，这样当然更会影响历史表启发，导致“历史表信息污染”。

为了不让“历史表信息丢失”又不让“历史表信息污染”，本系统尝试采用如下方法：不清空历史启发表，但每进行一次新的搜索都让历史启发表每项的历史值降低为原来的 $1/2^n$（n 为整数）。希望这样可以保留大部分的“历史启发信息”又避免“历史信息污染”。

但在实验中发现，每走一步直接清空历史启发表比衰减历史启发表信息要好（具体实验数据参看表 6.12 和表 6.13）。

6）迭代深化组合的优化

前面已经介绍，迭代深化可以在浅层获得启发信息。且前边的综合算法迭代深化时使用从第一层逐层递增，是否可以不从第一层开始，并且递增的层数（步

中国知网 https://www.cnki.net

长）大于1呢？在实验中发现答案是肯定的！

在实验时发现一条规律规律：当搜索深度、迭代的初始深度、迭代的递增量和能够迭代的次数满足下面的公式和条件：

搜索深度 =(迭代的开始深度 + 迭代的递增量*迭代次数)

(条件: (1)迭代的递增量>1; (2)迭代的开始深度+迭代的递增量 > 搜索深度)

时搜索算法比从第一层逐层递增好（现在还只在本文所研究的这个算法前九层验证过），并且在所做的实验中发现前八层除第六层外此公式包含每一层所有组合中的最优情况。

搜索深度为 n 的(迭代的初始深度，迭代的递增量)组合数为 $n(n-1)/2$ 种，而满足上面公式和条件时在第八层才三种。故利用可以上面公式和条件加快找到一种较好组合。

本文认为满足此公式时置换表的命中率提高、冲突减少，同时历史启发表的启发信息更准确。实验中七层的最优组合(3，2)比从第一层逐层递增提高12%的效率，八层的最优组合(4，2)比从第一层逐层递增提高16%的效率（具体实验数据参看表6.14和表6.15）。

7）迭代深化的时间控制

在时间控制的时候，前边的算法把时间判断放在搜索算法的外边，若搜索到n-1层时，时间已经快用完，再搜索第n层，搜索第n层所用的时间是第n-1层的5到10倍，要第n层搜索完才会停止搜索，这样使得时间控制很不准确（可能限定一分钟，实际搜索用5分钟以上）。本系统把时间控制放到搜索算法里边，这样就可以随时精确控制时间。同时由于迭代加深的原因，新的一层搜索的第一个着法总是上一层搜索得到的最佳着法，如果新的一层可以搜索出另一个更好的着法，就已经很满意了，有时没有必要找到最好的着法。换句话说，即使你没有足够的时间把这一层搜索完，得到的着法至少不会比上一层最好着法要坏，因此是可行的！

综上，从7个方面对综合算法进行了改进：动态设置置换表的大小和控制迭代深化的时间可以提高算法的实用性；使用双层置换表的覆盖策略、置换表不包含叶结点的估值以及衰减历史启发表信息没有取得算法效率上的提升；使用双层历史启发表和优化迭代深化的组合都能提高综合算法的效率。总的来说提高了实用性和效率（效率的具体实验数据参看表6.16和表6.17）。

中国知网 https://www.cnki.net

4.5 B*算法

上面介绍了以深度优先的$\alpha-\beta$搜索及其系列增强算法；另一类是最佳优先的系列算法[28]，包括仍然采用极大极小方法取值的SSS*算法[4, 29]和DUAL*算法，另一类是没有采用极大极小方法取值的B*和PB*算法。SSS*算法优于$\alpha-\beta$搜索[4],但与带置换表的$\alpha-\beta$搜索算法是等效的[21]。B*算法的研究比较少，国内资料中，只从陆汝钤老师的书[4]上看到比较全面的描述。

B*算法是由Hans J. Berliner在1979年提出来的一个算法[32, 33]，在1979年7月15日，Hans J. Berliner用此算法实现的十五子棋以7：1的悬殊比分击败了当时的世界冠军Luigi Villa，有史以来第一次在具有相当智力的棋类比赛中计算机程序击败了世界冠军[4]！ B*算法是到目前为止最具有人类风格的棋手[30]。

B*算法不限搜索深度，为了找到最佳走法，不必求根节点每个后继节点精确的极小极大值。每个节点用一个乐观估值和一个悲观估值来表示评价值。两个估计值都动态可变，且估值出自同一方的立场，只是估计的棋局按层次交错更替。对对方棋局的乐观估计即是对本方棋局的悲观估计；对对方棋局的悲观估计即是对本方棋局的乐观估计。因此，从下层节点向上层节点反馈信息时，悲观估计和乐观估计是交叉传递的。B*树在展开过程中，只要子节点的估值有利于父节点估值的精化，即改动父节点的估计值，即使乐观估值和一个悲观估值相互靠近(当当前节点深度大于零时需回溯)，如果这种改动波及到父节点的估值，则根节点需考虑使用何种策略。B*算法设立两种策略[31]，证明最优(PROVEBEST)和排除其余(DISPROVEREST)。算法从这点出发，用这两个界来证明哪个节点是最好的。当根节点的一个孩子的悲观值不比所有其它节点的乐观值差的时候，B*算法就结束了。这是因为此孩子节点对己方的有利程度大于等于所有其它孩子节点对己方的不利程度！也即是走这一枝赢的概率更大（这为本文研究得出的结论）！

算法的搜索控制就是尽可能快的得到终止条件。

B*算法的缺点是它对局面的乐观值和悲观值的估计依赖性太强。

PB*算法就是基于概率的B*算法，由Palay在1983年提出。这个算法对概率的准确估计比较敏感，算法在找到一个枝明显比其它枝好的时候停止搜索，但实现困难。

B*算法伪代码：

```
struct statue                          int opt; //棋局乐观估计值
{ qiju con; // qiju 为当前棋局内容(数组)   int pes; //棋局悲观估计值
```


中国知网 https://www.cnki.net

```
  unsigned par; //父节点所在的地址
  unsigned eld;//长子节点所在的地址
  unsigned young;幼子节点所在的地址
} sta;
sta = { cur_qiju, -ININITY, ININITY, 0, 0,
  0 };
qiju BXin(void)
{ vector< statue >   v;
  unsigned al = 1;   //第一个可分配的位置
  unsigned depth = 0; //当前搜索深度（层次）
  unsigned cur = 0;   //当前节点的地址
  statue   st;
  v.push_back(sta);
  while(1)
  { if(v[cur].eld == 0) //当前节点未扩展过
    { int count = CreatePossibleMove();
      for（each possibly move m）
      { make move m //产生 m 的子节点
        //创建子节点的状态
        construct child's STATUE st;
        v.push_back(st);
      }
      v[cur].eld = al; //修改长子节点所在地
  址
      //修改幼子节点所在的地址
      v[cur].young = al + Count - 1;
      al += count; //修改第一可分配的位置
    }
    //建立最佳节点和次佳节点
   lab:   best = next = pest = v[cur].eld;
     if(depth%2 == 0) //偶数层
      for(i = v[cur].eld; i <=v[cur].young;
i++)
      { if(v[best].opt < v[i].opt)
          best=i;//修改极大乐观值节点位置
       if(v[next].opt < v[i]. opt t &&
          v[i].opt < v[best]. opt)
        next=i;//修改次极大乐观值节点位置
       if(v[pest].pes < v[i]. pes)
         pest = i; //修改极大悲观值节点位置
      }
     else 为奇数层//
     for(i = v[cur].eld; i <=v[cur].young; i++)
     { if(v[best].pp.first > v[i].pp.first)
         best = i; //修改极小乐观值节点位置
       if(v[next].opt>v[i].opt&&
         v[i].opt> v[best]. opt)
        next=i;//修改次极小乐观值节点位置
       if(v[pest]. pes > v[i]. pes)
         pest = i; //修改极小悲观值节点位置
     }
     int opt = v[best].opt;
     int pes = v[pest].pes;
     //满足条件,则修改当前节点的估计值，
     并在深度大于零时进行回溯
     if( ((depth%2 == 0) && (opt <
      v[cur].pes || pes > v[cur].opt))
      || ( (depth%2 == 1) && (opt >
       v[cur].pes || pes < v[cur].opt)) )
     { v[cur].pes = opt;
       v[cur].opt = pes;
       if(depth > 0)
```


中国知网 https://www.cnki.net

```
{ cur = v[cur].par;
 depth--;
 goto lab; //回溯
}
else
//有一枝的悲观值不小于其它枝的乐
观值，则搜索结束。
 if(v[best].pes >= v[next].opt)
     return v[best].con;
 }
 if(depth == 0)//决定下一步的策略
 { if(take provebest strategy)   cur = best;
   else   cur = next;
 }
 if(depth > 0)   cur = best;
 depth++;
}//end of while
}//end of BXin
```

实验中用的估值很简单，设站在某一方对棋局的估计值为 value，m>1,0<n<1,则对本方的乐观估计为 m×value;悲观估计为 n×value; 对对方的乐观估计为 n×value;悲观估计为 m×value;用的策略为 Berliner 原则：用一组候选分枝与最佳分枝做比较，如果各候选分枝实行信息反馈的的深度是 ti,最佳分枝实行信息反馈的的深度是 t,比较$\sum ti^2$和 t^2,若前者小于后者，则采用 DISPROVEREST 策略，否则采用 PROVEBEST 策略。

中国知网 https://www.cnki.net

第五章 系统设计

本系统运用面向对象（OO）方法对系统进行分析与设计，并用统一建模语言（UML: Unified Modeling Language）对系统进行建模进行描述，最后使用 C++ 语言在 Microsoft Visual C++ 6.0 上实现系统。

5.1 面向对象的分析与设计简介

面向对象方法是一种运用对象、类、封装、聚合、消息传递、多态性等概念来构造系统的软件开发方法[34]。基本概念介绍如下：

1)对象: 系统中用来描述客观事物的一个实体，它是构成系统的基本单位。它由属性和对属性进行操作的服务构成（变量）。

2)类: 具有相同属性和服务的一组对象的集合，它为该类的全部对象提供了统一的抽象描述（类型）。

3)封装: 把对象的属性和服务结合成一个独立的单元，尽可能隐蔽对象的内部细节（实现），为用户提供方便可用的接口，可以避免交叉感染（易于调试），波动效应（便于修改）。

4)继承: 子类拥有父类的全部属性和服务（传递关系）。

5)消息: 指向对象发出的服务请求，它有提供服务的对象标识，服务标识，输入信息和回答信息（函数调用）。

6)结构和连接: 包括一般－特殊，整体－部分（嵌套对象，属性对象），实例连接（静态关系），消息连接（动态关系）。

7)多态性: 指父类的中定义的属性和服务在子类可以有不同的数据类型或行为（重载和动态绑定）。

8)永久对象: 生存期可以超越程序的执行时间而长期存在的对象（可用数据库系统实现）。

9)主动对象: 至少有一个服务不需要接收消息就可以主动执行的对象（进程和线程）。

10)系统边界: 指一个系统所包含的所有系统成份与系统以外的分界线，其中系统是指计算机软硬件系统,与开发者的责任有关 ，系统成分是指那些在 OOA 和 OOD 中定义并且在编程时加以实现的系统元素 (类和对象) 。

中国知网 https://www.cnki.net

11)活动者：指在系统之外与系统进行交互的任何事物，如:人员,设备,外系统(研究他是为了搞清系统对他的外部世界所表现的行为).

面向对象分析（OOA）就是运用面向对象方法（进行需求分析）,对问题域和系统责任进行分析和理解,对其中的事物和他们之间的关系产生正确的认识，找出描述问题和系统责任所需要的类和对象，定义这些类和对象的属性与服务，以及他们之间所形成的结构及静态和动态关系．产生一个符合用户需求，并能直接反映问题域和系统责任的OOA模型及详细说明．OOA的主要优点有：可以加强了对问题域和系统责任的理解（无缝连接）；改进分析人员和各类人员的交流；对需求具有更强的适应性；满足软件复用要求。

OOA 模型包括基本模型--类图及补充模型--用例图，主题图和交互图，具体的建模过程可参看[34]，里面有详细的过程指导。由于本系统将比较简单，只实现了类图。

面向对象设计（OOD）就是运用面向对象方法进行系统设计,它是基于OOA的,与OOA有一致的概念和表示方法。OOD的主要工作内容是以OOA为模型和基础，按照实现的要求进行设计和决策，包括全局的决策和局部细节的设计，目标是产生一个满足用户需求，并且完全可实现的OOD模型[35]。

OOD 模型包括：以 OOA 模型作为输入，根据实现要求对 OOA 模型作必要的修改与调整，或补充某些细节的问题域部分；根据具体的界面支持系统（窗口系统、可视化编程环境）而设计的人机交互部分；根据具体的硬件、操作系统和网络设施而设计的控制驱动部分；根据具体的数据管理系统（如文件系统或数据库管理系统）而设计的数据接口部分。具体的建模过程可参看[35]，里面有详细的过程指导。

由于本系统比较简单，主要需对问题域部分的OOA模型进行修改、调整与细化；而在人机交互部分直接在可视化编程环境中加入合适的元素即可完成；对控制驱动部分，由于程序是单机版，且在单CPU的PC机上运行，所以对于系统的总体方案、体系结构、程序并发性考虑较少；对数据接口部分，由于数据在程序开始就都保存在内存中（数组中），故暂不用考虑这个部分。

对系统建模后，需用大家能够理解的建模语言把它描述出来，以便于人员之间的交流，这很重要。下面介绍统的统一建模语言（UML）能有效解决这个问题。

5.2 UML 简介

UML是一种建模语言(modeling language)，用于对软件系统制品进行详述、

中国知网 https://www.cnki.net

可视化、构造和文档化；也可以用于业务建模及其他非软件系统的建模[35]。作为一种建模语言，UML定义了面向对象系统模型所需要的概念（建模元素）并给出其可视化表示；但不涉及如何建模。

UML包括概念的语义，表示法和说明，提供了静态、动态、系统环境及组织结构的模型。它可被交互的可视化建模工具所支持，这些工具提供了代码生成器和报表生成器。UML标准并没有定义一种标准的开发过程，但它适用于迭代式的开发过程。它是为支持大部分现存的面向对象开发过程而设计的。UML描述了一个系统的静态结构和动态行为。

UML将系统描述为一些离散的相互作用的对象并最终为外部用户提供一定的功能的模型结构。其中静态结构定义了系统中的重要对象的属性和操作以及这些对象之间的相互关系。动态行为定义了对象的时间特性和对象为完成目标而相互进行通信的机制。从不同但相互联系的角度对系统建立的模型可用于不同的目的。

UML还包括可将模型分解成包的结构组件，以便于软件小组将大的系统分解成易于处理的块结构，并理解和控制各个包之间的依赖关系，在复杂的开发环境中管理模型单元。它还包括用于显示系统实现和组织运行的组件。

UML不是一门程序设计语言。但可以使用代码生成器工具将UML模型转换为多种程序设计语言代码，或使用反向生成器工具将程序源代码转换为UML；UML不是一种可用于定理证明的高度形式化的语言，这样的语言有很多种，但它们通用性较差，不易理解和使用。UML是一种通用建模语言。对于一些专门领域，例如用户图形界面（GUI）设计、超大规模集成电路（VLSI ）设计、基于规则的人工智能领域，使用专门的语言和工具可能会更适合些。

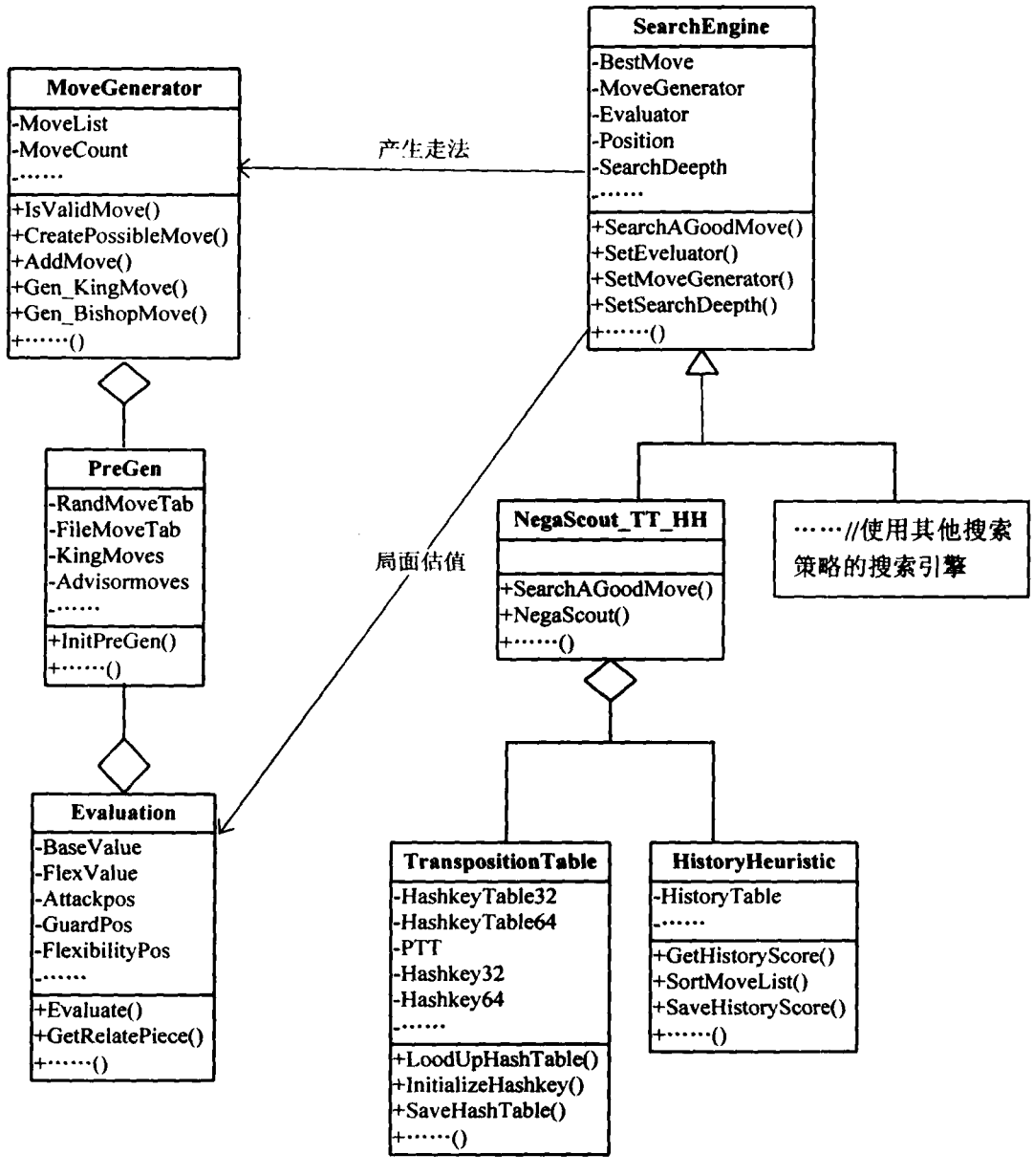
UML是一种离散的建模语言，不适合对诸如工程和物理学领域中的连续系统建模。它是一个综合的通用建模语言，适合对诸如由计算机软件、固件或数字逻辑构成的离散系统建模。

UML是一种定义良好、易于表达、功能强大且普遍适用的建模语言。它溶入了软件工程领域的新思想、新方法和新技术。它的作用域不限于支持面向对象的分析与设计，还支持从需求分析开始的软件开发的全过程。

UML提供了五类图(共9种图形)来实现建模过程[36]。下面列出最基本的类图在本系统中的实现。各种建模元素的可视化遵循邵维忠，杨芙清老师的《面向对象系统设计》一书上面的表示。

中国知网 https://www.cnki.net

5.3 类图



从上面的类图可以看出：各模块之间独立性很高，这有利于模块的复用和系统的扩充；同时，通过类图可以对系统的整体框架有更清楚的把握。

5.4 系统的实现及用户界面操作

系统的具体实现使用C++语言[37,38]在Visual C++ 6.0的集成开发环境[41]上面完成。在数据结构上使用了C++ STL的vctor容器[39]，MFC的相关类[40]。

在界面上可以从菜单设定先后手， 悔棋， 换色，搜索算法及算法的策略设置，也可以让机器之间对弈。悔棋时最多能悔三步棋：如果您已经悔了n（n<=3）步棋,您又走了m步棋，则您还能悔x步棋，其中x=(3+m-n)>=3 ? 3 : (3+m-n)。

中国知网 https://www.cnki.net

第六章 实验与结果

6.1 程序说明

本论文涉及三个博弈程序：第一个博弈程序是基于10×9的棋盘数据结构，没有加入走法预生成，它的实现基于王小春的《游戏编程<人机博弈>》里的象棋程序，不过为了实验需要加入了机器对弈模块，综合算法，同时也包括界面的一些功能；第二个博弈程序是本论文的实现。第三个博弈程序使用B*算法，用的是第一个程序的框架，但估值不一样。

之所以实现三个程序，主要是为了从数据结构，走法预生成，算法等方面进行比较。对第三博弈个程序的研究还不成熟，作为本文的一部分是因为在国内对B*算法的研究比较少，国内资料中，只从陆汝钤老师的书[4]上看到比较全面的描述，做为兴趣与锻炼实现了！

6.2 实验结果及分析

说明：用于实验的机器配置为 Pentium4 CPU 2.8GH,512MB 内存。表 6.2 到表 6.17 的实验数据是在无人干涉的条件下被测试的搜索引擎与另一固定的搜索引擎在开局双方各走十步获得的结果（另一固定搜索引擎的搜索深度为 5，它的搜索深度不同时实验数据不一样：它的搜索深度越深，被测试的搜索引擎的效率略有降低）。

1. 首先考察不同的棋盘表示以及是否加入走法预生成模块对走法产生和局面估值的影响，如表 6.1。

表 6.1 在不同棋盘表示下对初始局面进行一百万次操作所用的时间(ms)

操作 / 棋盘表示	走法产生	局面估值
10×9棋盘结构	8 520	25 017
行位列扩展棋盘+走法预生成	4 236	15 504

可以看到通过改进棋盘，使用走法预生成模块可以很大程度提高走法产生和局面估值的效率：走法产生提高了 50%，局面估值提高 38%。

2. 考察综合算法在不同棋盘表示下的效率，如表6.2与表6.3。

中国知网 https://www.cnki.net

表6.2 综合算法在不同棋盘表示下开局十步平均每步所用的时间(ms)

层数 / 棋盘表示	1	2	3	4	5	6	7	8
10×9棋盘结构	<1	<1	20	73	487	5 553	23 014	139 723
位行位列扩展棋盘+走法预生成	<1	<1	7	29	229	2 537	9 018	61 203

可以看到在使用带位行位列信息的扩展棋盘加上走法预生成模块极大的提高了搜索的效率：在10×9棋盘结构上提高2到3倍。

表6.3 综合算法在不同棋盘表示下开局十步平均每步评估的节点数

层 数 / 数据结构	1	2	3	4	5	6	7	8
10×9棋盘结构	41	213	2 059	6 396	41 834	480 676	1 856 316	11 567 660
位行位列扩展棋盘+走法预生成	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070

从表中可以看到不同棋盘表示每步评估的节点数基本一致，这说明效率的提高是从走法产生和局面估值部分获得的。

下面综合算法都基于带位行位列信息的扩展棋盘加上走法预生成模块。

3. 接着考察改进综合算法时使用不同策略的实验数据。

(1) 设置的置换表的大小不一样时，算法的效率如表6.4和表6.5

表6.4 置换表大小不同时开局十步平均每步所用的时间(ms)

层数 / 置换表大小	1	2	3	4	5	6	7	8
6	<1	3	6	34	231	2 246	10 449	77 768
12	<1	3	9	35	231	2 232	10 040	74 498
24	<1	1	10	35	232	2 173	9 225	74 236,
48	<1	<1	7	29	229	2 537	9 018	61 203
96	<1	1	7	34	235	2 279	8 578	61 698
192	<1	<1	7	32	239	2 431	8 567	57 173

中国知网 https://www.cnki.net

从表6.4中可以看到搜索到六层时6MB的置换表已经够用，7层时96MB的置换表是比较合适的。增加一层，需要置换表需要增加的大小幅度不是线性的。

表6.5 置换表大小不同时开局十步平均每步评估的节点数

层数 / 置换表大小	1	2	3	4	5	6	7	8
6	41	220	1 777	6 114	43 127	406 742	1 853 514	14 347 361
12	41	220	1 774	6 092	42 804	394 690	1 750 743	13 730 658
24	41	220	1 774	6 235	42 486	376 266	1 602 281	13 614 791
48	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070
96	41	220	1 774	6 082	43 99	397 106	1 474 228	11 070 304
192	41	220	1 773	6 082	43 179	421 428	1 455 594	10 078 637

注：置换表的大小以MB为单位

从表6.5可以发现，评估的节点数在六层时，6MB的置换表发生的冲突已经很小了，所以置换表增大对效率提高不大；7层时，则需要96MB的置换表。当然从表中可以看到使用置换表提高效率的原因：在发生冲突时随着置换表的增大评估的节点数少了！

下面综合算法的置换表的大小皆为48MB。

(2) 置换表的覆盖策略不同时，算法的效率如表6.6和6.7。

表6.6置换表的覆盖策略不同时开局十步平均每步所用的时间(ms)

层 数 / 置换表覆盖策略	1	2	3	4	5	6	7	8
始终覆盖	<1	<1	7	29	229	2 537	9 018	61 203
深度优先	<1	<1	10	35	232	2 656	10 309	91 867
双层置换表	<1	<1	9	34	232	2 534	9 421	64 423

从表6.6中可以看到，使用始终覆盖优于深度优先和双层置换表。并且随着深度的增加，深度优先表现越来越糟糕，这说明深度优先覆盖产生更多的冲突：某个节点的估值刚被替换掉，马上又要被使用，此时必须重新估值。

中国知网 https://www.cnki.net

表6.7 置换表的覆盖策略不同时开局十步平均每步评估的节点数

层 数 / 置换表覆盖策略	1	2	3	4	5	6	7	8
始终覆盖	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070
深度优先	41	220	1 774	6 102	42 670	481 177	1 892 783	18 937 833
双层置换表	41	220	1 774	6 099	42 243	432 737	1 580 703	11 474 435

从表6.7中可以看出不同覆盖策略产生差异的原因：使用深度优先时，置换表的冲突比使用始终覆盖大！

下面综合算法的置换表使用始终覆盖策略。

(3) 置换表对叶结点的处理不同时，算法的效率如表6.8和表6.9

表6.8 置换表对叶结点处理不同时开局十步平均每步所用的时间(ms)

层 数 / 置换表包含叶结点	1	2	3	4	5	6	7	8
是	<1	<1	7	29	229	2 537	9 018	61 203
否	<1	<1	12	46	338	3 574	11 536	82 357

从表6.8中可以看出，置换表不包含叶结点估值不如包含叶结点的估值。但如果在机器内存十分有限的时候，不能使用大的置换表时，置换表不包括叶结点估值是可取的。

表6.9 置换表对叶结点处理不同时开局十步平均每步评估的节点数

层 数 / 置换表包含叶结点	1	2	3	4	5	6	7	8
是	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070
否	45	281	2 630	10 411	76 588	805 009	2 464 191	17 755 999

从表中可以看出，置换表不包括叶结点时，每步评估的节点数大幅度上升。由于当前的估值部分所耗时间比从置换表直接取估值多，置换表包括叶结点优势更明显。

下面综合算法的置换表包括叶结点的估值。

(4) 历史启发表的层数不同时，算法的效率如表6.10和表6.11

中国知网 https://www.cnki.net

表6.10 历史启发表的层数不同时开局十步平均每步所用的时间(ms)

层数 / 历史启发表层数	1	2	3	4	5	6	7	8
单层	<1	<1	7	29	229	2 537	9 018	61 203
双层	<1	1	7	32	246	2 445	8 390	49 448

从表6.10中可以看到，在前五层使用双层历史启发表略差于单层历史启发表，这是因为访问双层历史启发表比访问单层历史启发表花费的时间多；到六层以后使用双层历史启发表开始优于单层历史启发表，这是因为双层历史启发表的信息更加准确，更有利于剪枝，它可以抵消因访问双层历史启发表所花费的时间。

表6.11 历史启发表的层数不同时开局十步平均每步评估的节点数

层数 / 历史启发表层数	1	2	3	4	5	6	7	8
单层	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070
双层	41	220	1 773	5 885	45 456	421 451	1 493 668	8 563 569

从表6.11中可以看出前五层双层历史启发表与单层历史启发表所估节点数不相上下，六层以后双层历史启发表比单层历史启发表所估节点数明显减少，这是由于双层历史启发表的信息更加准确，更有利于剪枝。

下面综合算法的使用双层的历史启发表。

(5) 历史启发表的衰减度不同时，算法的效率如表6.12和表6.13

表6.12 历史启发表的衰减度不同时开局十步平均每步所用的时间(ms)

层数 / 衰减度(n)	1	2	3	4	5	6	7	8
始终清空(+∞)	<1	1	7	32	246	2 445	8 390	49 448
1	<1	<1	9	47	406	7 125	10 579	69 095
2	<1	3	9	47	390	6 250	9 186	63 576
3	<1	<1	6	47	273	3 220	9 325	60 481
4	<1	<1	11	37	273	3 184	8 690	57 512

注：衰减度 n 表示把历史启发表中的历史启发值变成原来的 $1/2^n$

从表6.12中可以看出直接清空历史启发表比对历史启发表信息进行衰减要

中国知网 https://www.cnki.net

好，这出乎实验设计的意料之外，本来认为通过保留部分历史启发表信息应该能提高搜索效率。这应该是历史期发表的“历史信息污染”使得剪枝效率降低！

表6.13 历史启发表的衰减度不同时开局十步平均每步评估的节点数

层数 / 衰减度(n)	1	2	3	4	5	6	7	8
始终清空(+∞)	41	220	1 773	5 885	45 456	421 451	1 493 668	8 563 569
1	41	228	1 778	7 193	48 397	644 685	1 921 536	12 184 818
2	41	225	1 709	6 656	44 898	612 654	1 658 631	11 136 455
3	41	215	1 736	6 212	48064	557 679	1 700 163	10 569 063
4	41	218	1 773	7 000	48 632	557 306	1 567 425	10 015 859

从表中可以看到保留越多的历史启发信息使得估值的节点数反而增加，剪枝的效率下降！“历史启发信息”的效用还不如“历史信息污染”的损耗。

下面综合算法每走一步直接清空双层历史启发表。

(6)迭代深化的初始深度与迭代的递增量组合不同时，算法的效率如表6.14和表6.15

表6.14 迭代深化所用组合不同时开局十步平均每步所用的时间(ms)

组合 / 层数	3	4	5	6	7	8
1-1	7	32	218	2 242	8 015	49 173
1-2	12	68	**192**	1 432	7 853	61 226
1-3	—	31	210	**1 060**	7 831	51 467
1-4	—	—	253	1 797	7 353	64 895
1-5	—	—	—	2 654	17 678	112 878
1-6	—	—	—	—	12 498	74 982
1-7	—	—	—	—	—	56 792
2-1	**6**	31	221	2 087	8 240	52 378
2-2	—	**26**	195	2 029	14 776	47 648
2-3	—	—	195	1 787	7 686	42 834
2-4	—	—	—	2 479	15 448	100 593
2-5	—	—	—	—	9 768	73 086
2-6	—	—	—	—	—	64 981
3-1	—	32	218	2 376	8 392	46 768

中国知网 https://www.cnki.net

3-2	—	—	198	1 517	**7 020**	50 139
3-3	—	—	—	2 004	14 228	87 497
3-4	—	—	—	—	7 759	53 888
3-5	—	—	—	—	—	46 045
4-1	—	—	232	2 240	7 548	45 454
4-2	—	—	—	2017	13 451	**40 953**
4-3	—	—	—	—	7 360	54 801
4-4	—	—	—	—	—	42 370
5-1	—	—	—	2 726	7 500	51 351
5-2	—	—	—	—	7 671	55 108
5-3	—	—	—	—	—	43 998
6-1	—	—	—	—	8 885	50 064
6-2	—	—	—	—	—	44 350
7-1	—	—	—	—	—	48 446

注：加粗的栏为最优的组合，‘—’表示在此层数上没有此组合。

从表中可以看出，凡是符合：搜索深度 = 迭代的初始深度 + 迭代的递增量×迭代次数) 并且满足 (1)迭代的递增量>1; (2)迭代的初始深度+迭代的递增量 > 搜索深度则比从第一层逐层递增好；除第六层外，符合规律的包含最优的组合情况。这对最优组合的选择可以提供帮助！而对不符合规律的，和从第一层逐层递增比有好有坏。

表6. 15 迭代深化所用组合不同时开局十步平均每步评估的节点数

组合 \ 层数	3	4	5	6	7	8
1-1	1 773	5 620	39 404	378 476	1 422 964	8 415 327
1-2	1 802	12 867	**34 951**	247 560	1 416 578	10 896 458
1-3	—	5 464	37 341	**180 799**	1 414 130	8 955 941
1-4	—	—	46 656	310 779	1 256 918	11 620 801
1-5	—	—	—	489 137	3 090 095	21 048 588
1-6	—	—	—	—	2 334 445	13 181 797
1-7	—	—	—	—	—	9 828 079
2-1	**1 747**	5 822	40 511	355 801	1 466 649	9 087 332
2-2	—	**4 829**	34 182	359 207	2 533 401	8 258 576

中国知网 https://www.cnki.net

2-3	—	—	36 553	311 161	1 354 525	7 499 818
2-4	—	—	—	451 011	2 665 582	18 599 190
2-5	—	—	—	—	1 800 525	12 879 604
2-6	—	—	—	—	—	11 393 698
3-1	—	5 872	39 742	411 016	1 487 129	7 968 041
3-2	—	—	35 721	262 735	**1 263 989**	8 817 938
3-3	—	—	—	354 506	2 441 520	16 196 595
3-4	—	—	—	—	1 434 874	9 400 241
3-5	—	—	—	—	—	8 039 779
4-1	—	—	42 775	386 515	1 334 331	7 777 686
4-2	—	—	—	354 728	2 305 879	**7 081 041**
4-3	—	—	—	—	1 344 736	9 654 507
4-4	—	—	—	—	—	7 339 908
5-1	—	—	—	472 379	1 341 653	8 927 049
5-2	—	—	—	—	1 400 444	9 753 241
5-3	—	—	—	—	—	7 698 756
6-1	—	—	—	—	1 562 361	8 622 349
6-2	—	—	—	—	—	7 672 801
7-1	—	—	—	—	—	8 451 802

从表中可以看到，在好的组合下估值的节点数少。本文认为这是因为在好的组合下使得置换表的冲突减少、命中率提高，同时历史期发表的信息更准确。这使得搜索算法效率提高。

(7)改进后的综合算法与原综合算法比较，算法的效率如表6.16和表6.17

表6. 16综合算法与改进的综合算法开局十步平均每步所用的时间(ms)

层数 / 算法	1	2	3	4	5	6	7	8
综合算法	<1	<1	7	29	229	2 537	9 018	61 203
改进的综合算法	<1	<1	6	26	192	1 060	7 020	40 953

注：原综合算法的置换表的大小为48MB、采用始终覆盖策略、包括叶结点估值，历史启发表使用单层、每走一步直接清空历史启发信息，迭代深化是从第一层逐层递增的；改进

中国知网 https://www.cnki.net

后的综合算法的置换表的大小为48MB、采用始终覆盖策略、包括叶结点估值，历史启发表使用双层、每走一步直接清空历史启发信息，迭代深化是使用迭代的最佳组合。

从表中可以看出改进后的综合算法在原来的综合算法基础上效率提高了：第五层提高了16%，第六层提高了58%，第七层提高了22%，第八层提高了33%。

表6.17综合算法与改进算法开局十步平均每步评估的节点数

层数 / 算法与数据结构	1	2		4	5	6	7	8
综合算法	41	222	1 773	6 111	42 537	444 515	1 558 296	11 089 070
改进算法	41	220	1 747	4 829	34 951	180 799	1 263 989	7 081 041

从表中可以看出，改进的综合算法比原综合算法评估的节点数减少了，这是由于使用双层历史启发表与优化迭代深化时的组合使得启发表信息更加准确，提高了剪枝效率；同时优化迭代深化时的组合使得置换表的冲突减少、命中率提高。

同时，实验中还发现开局开始时走的时间偏长，估计的节点较多，越往后时间一般越短，节点评估越少，这样如果利用开局库能提高搜索算法的实用性！

4. 最后考察B*算法的实验结果

B*算法速度很快，实验所用的估值效果实现算法速度很快：时间小于1ms，估值节点不多于100个；但由于B*算法对估值的依赖性很强，走法只有搜索深度为4层$\alpha-\beta$的水准。如何在此算法下进行合理估值有待进一步研究。

中国知网 https://www.cnki.net

第七章 总结与展望

7.1 工作总结

本文对中国象棋计算机博弈技术进行了研究，在此基础上，实现了一个具有一定棋力的中国象棋博弈系统。在实现中采用了前人的研究成果，并在很多次的错误、失败和成功后，对前人的理论和方法有了深刻的理解，明白了其中的原理，对如何开发计算机下棋程序有了自己实际的经验。

在系统的设计实现过程中，受益颇多：

1.理解了数据结构的重要性。最初的系统使用10×9的棋盘数据结构，它在使用综合算法下搜索7层需要23秒，但使用带位行位列信息的扩展棋盘，同时使用走法预生成（以空间换时间），搜索效率明显改善，搜索7层可以控制在10秒以内!

2.体会最深刻的是对搜索算法的研究。做的象棋程序第一个版本采用$\alpha-\beta$搜索算法，最终系统采用基于迭代加深、置换表、历史启发的NegaScout改进算法。相同数据结构下比较两个算法,在五层以上同层相比：时间降低两个数量级以上，搜索的节点数降低三个数量级以上，且层数越大效果越明显。这充分体现了算法的魅力。

3.还有一个体会是要有先进的软件设计理念，如果一开始系统搭得不好，以后的每一步改动都会很麻烦。本系统采用面向对象的体系结构，这个结构最大化的优点是：有利系统的复用，各模块之间独立性强，某一模块的改变只需要修改其他模块的很小部分甚至不用修改。如估值部分变了，其他部分不需要做任何修改。

4.让计算机帮你做尽可能多的事。系统设计之初，为了统计搜索的效率和性能，使用人工方式求平均节点数和平均时间，速度慢且性能方面也不好评判！后来在程序中加入机器对弈模块，在机器对弈过程中统计搜索的效率和性能的评判（能赢棋性能当然更好），这极大提高效率。

7.2 今后的工作

本系统还有很多地方有待完善，下面提出下一阶段的研究设想。

1. 对搜索算法的研究仍有待于更加深入，加入更多地搜索策略，在对基于$\alpha-\beta$剪枝搜索算法要用更多的具体棋类知识来指导搜索。使得搜索效率进一步

中国知网 https://www.cnki.net

提高，同时保证有前途的几枝不被剪掉。进一步研究B*算法，它没有搜索深度的限制！

2．改进的估值函数并使之与搜索算法的有机结合。本系统采用的是传统的静态估值函数，对局面位置特征的评价并不完善，对局势的好坏判断不够灵敏。由于估值函数是与具体的棋类知识紧密结合的一部分，所以可以在估值函数中加入更多的象棋知识，使之对于局面的评估更为精确，但也要做好程序的简化，不能因为精确而使速度下降过大。另外由于B*算法对估值非常敏感，可能合理的估值结合B*算法可以很大程度提高程序的效率与性能。

3.增加开局库和残局库。把开局几步的走法建成数据库供程序直接取用，可以大大加快开局时的运行速度和提高开局的质量。使用残局库可以大大加快象棋后的棋力。

4.加入机器学习模块。机器学习可以优化估值函数，丰富模式库中的内容，提高程序的博弈性能。当前东北大学的“齐天大圣”象棋软件就加入了机器学习模块。如何在自己的象棋程序中加入机器学习将是一大挑战。

虽然本文实现了一个具有一定棋力中国象棋博弈系统，但由于中国象棋本身的复杂性，以及时间和作者的水平有限，文中存在一定的不足之处，敬请各位专家、读者批评指正。

中国知网 https://www.cnki.net

致谢

首先我要感谢的是我的导师王海瑞老师。在两年多的学习生活中他教给了我很多的学习方法，同时向我展现了很多的学习领域，可以说在这几年的学习中是我知识面扩展最大的几年。我做论文期间，王老师教导我学术研究的方法，从选题直至成稿一直给予我重要的指导和帮助，为我解开了很多的困惑，提供了很多关键性的建议。可以说我在这里取得的成果是和王老师的教育密切相关的，在此对王老师表示衷心的感谢！

然后我要衷心的感谢我的各位任课老师。是他们的教导使我获得扎实的专业知识。我要对老师们致以深深的敬意！

还有我需要感谢的是实验室的所有的同学，他们给了我很大的帮助。很幸运我们能够在学校里面相识，在一起学习的日子里，同时也结下了深厚的友谊，这些都是我宝贵的财富。希望在未来的人生道路上，我们还能继续互相鼓励，共同进步。还要感谢我的同学，在论文撰写和编写程序上给我的建议和帮助。

最后要感谢是我的家人。他们殷切的期望和无尽的关怀是我不断进取的动力，正是因为他们无私的帮助和鼓励，我才走到了今天!我对他们表示衷心的感谢！

感谢所有曾经帮助和关心我的人，谢谢你们！

中国知网 https://www.cnki.net

参考文献

[1] 徐心和，王骄. 中国象棋计算机博弈关键技术分析. 小型微型计算机系统. 2006年，第27卷第6期：961-969

[2] 徐心和，王骄. 向中国象棋冠军发起挑战——广泛深入开展计算机博弈研究. 中国人工智能学会第11届全国学术年会论文集 2000年9月

[3] 许舜钦. 电脑西洋棋和电脑象棋的回顾与前瞻. 电脑学刊(台)1990, 3(2):1-8

[4] 陆汝钤. 人工智能（上册）. 第一版. 北京：科学出版社，1989. 8

[5] 王小春. PC游戏编程（人机博弈）. 第一版. 重庆：重庆大学出版社 ，2002. 6

[6] 肖齐英，王正志. 博弈树搜索与静态估值函数. 计算机应用研究. 1997，4:74-76.

[7] 张赜. 计算机中国象棋博弈中的二次估值方法及其优化研究：[硕士学位论文]. 吉林：东北大学，2006

[8] Marsland T A. Computer chess and search [D]. Edmonton :University of Alberta, 1991

[9] Neill Graham. Artificial Intelligence-Make machines "think". TAB BOOKS, 1979

[10] TODHUN TER I. A History of the Mathematical Theory of Probability from the Time of Passcal to that of Lap lace [M] New York: Chelsea Publishing, 1965: 105-108.

[11] BOREL E. The theory of play and integral equations with skew symmetric kernels[J]. Econometrica, 1953,21: 91—117.

[12] Von N EUMANN J. Communication on the Borel notes [J] Econom et rica, 1953, 21: 124-125

[13] Rivest, R-L Game Tree Searching by MinMax Approximation. Aritficial Intelligence, 1998, Vol. 34, No. 1

[14] 王骐. 博弈树搜索算法的研究及改进. [硕士学位论文]. 浙江：浙江大学，2006

[15] Donald E, Knuth and Ronald W, Moore. An Analysis of Alpha-Beta Pruning Artificial Intelligence, vol.6，1975: 293-326

[16] Jonathan Schaeffer. The history heuristic and alpha-beta search enhancements in

中国知网 https://www.cnki.net

practice. IEEE Transactions on Pattern Analysis and Machine Intelligence, November 1989, PAMI-(1):1203~1212

[17]A.Reinefeld.Spielbaum-Suchverfahren.Informatik-Fachbericht 200,Spring-Verlag, Berlin(1989),ISBN 3-540-50724-6

[18] A. Zobrist. A New Hashing Method with Application for Game Playing Technical Report 88,Computer Science Department, University of Wisconsin, Madison. 1970

[19] Shannon，Claude E. Programming a computer for playing chess [J]. Philosophical Magazine，1950，41: 256—275

[20] Hermann Kaindl，Reza Shams.Helmut Horacek. Minimax search algorithms with and without aspiration windows[J]. IEEE Transactions Pattern Analysis and M achine Intelligence，December 1991，13(1Z) 1225—1235

[21] Aske Plaat,Jonathan Schaeffer,Wim Pijls,et al. SSS*=$\alpha-\beta$+TT.Canada:Technical Report TR-CS-94-17,Department of Computer Science,University of Alberta,Edmonton,AB,December 1994

[22] Aske Plaat, Jonathan Schaeffer, Wim Pijls, et al. 'A New paradingn for minimax search'. Canada: Technical Report TR-CS-94-18, Department of Computer Science, University of Alberta, Edmonton, AB, December 1994

[23] Aske Plaat, Jonathan Schaeffer, Wim Pijls, et al. 'Nearly optimal minimax tree search?'. Canada: Technical Report TR-CS-94-19, Department of Computer Science, University of Alberta, Edmonton, AB, December 1994

[24] Charles E. Leiserson and Aske Plaat.Programming Parallel Applications in Cilk, *SIAM News*, Volume 31, Number 4, May, 1998, pp 6-7.

[25] Don F Beal. A generalised quiescence search algorithm，department of computer science，queen mary college[J]. London Unlversity，Artificial Intelligence. 1990,43：85—98.

[26] Holland J H.Adaptation in nature and artificial system[M].Ann Arbor:The University of Michigan Press,1975.

[27] Richard S. Sutton (1988) Learning to predict by the methods of temporal difference. Machine Learning 3,9-44

[28] Arie de Bruin and Wim Pijls.Trends in Game tree Search.1996

中国知网 https://www.cnki.net

[29] A.Plaat, J.Schaeffer, W.Pijls, and A.de Bruin.Best-First Fixed-Depth Minimax Algorithms. Artificial Intelligence, 87(1-2):255~293, Nov.1996

[30] [美]Nils J Nilsson. 人工智能. 北京：机械工业出版社 ，2000.

[31] Palay, A.J. (1985).*Searching with Probabilities*.Pitman Publishers. Originally (1983) published as Ph.D. Thesis, Carnegie-Mellon University.

[32] Berliner, H.J. (1979). The B*-tree search: A best-first proof procedure. *Artificial Intelligence*, Vol. 12, No. 1, pp. 23-40.ISSN0004-3702.

[33] Berliner, H.J. and McConnell, C. (1996).B* probability based search. *Artificial Intelligence*, Vol. 86, No. 1, pp. 23-40.ISSN0004-3702.

[34] 邵维忠，杨芙清. 面向对象系统分析. 北京：清华大学出版社，1998

[35] 邵维忠，杨芙清. 面向对象系统设计. 北京：清华大学出版社，2003

[36] Joseph Schmuller著；李虎，赵龙刚(译).UML基础、案例与应用（第三版）. 北京：人民邮电出版社，2004

[37] 钱能，董灵平，张敏霞. C++程序设计. 北京：清华大学出版社，1999

[38] Bruce Eckel 著；刘宗田(译). C++ 编程思想. 北京：机械工业出版社，2004

[39] 彭木根，王淑凌. C++ STL程序员开发指南. 北京：中国铁道出版社，2003

[40] 王艳平. Windows 程序设计. 北京： 人民邮电出版社，2005

[41] David J.Kruglinski著；王国印(译). Visual C++技术内幕. 北京：清华大学出版社，1996

网络资源：

[42] Alpha-Beta search Bruce Moreland论述了$\alpha-\beta$剪枝算法 http://www.seanet.com/~brucemo/topics/alphabeta.htm

[43]Bruce Moreland 的国际象棋引擎，介绍博弈程序设计指南，开局库及源代码。http://www.seanet.com/~brucemo/chess.htm

[44]http://www.elephantbase.net， 电脑象棋百科全书，有大量有关象棋的知识技术

[45] http://www.elephantbase.net/computer/eleeye_intro.htm，黄晨对“象眼”引擎的分析

中国知网 https://www.cnki.net

攻读硕士学位期间发表的文章

1. 危春波，王海瑞，文乔农. 博弈树搜索算法的分析与实现. 江西科技广场（信息科学），2007.5
2. 文乔农，王海瑞，危春波. 车牌字符识别研究. 江西科技广场（信息科学），2007.5

中国知网 https://www.cnki.net